



universidad
de león

Escuela de Ingenierías Industrial, Informática y
Aeroespacial
Universidad de León

Matemática Finita II

Grado en Ingeniería de Datos e Inteligencia Artificial

BLOQUE I

Unidad 1: Análisis teórico de la complejidad computacional

Dr. Miguel Angel Navarro Burgos

Contenido I

Fundamentos del análisis de algoritmos

Notaciones asintóticas

- Notación O (cota superior)

- Notación Ω (cota inferior)

- Notación Θ (cota ajustada)

- Relación formal y aplicación práctica de notaciones asintóticas

Fundamentos de la notación asintótica

Tiempo vs espacio

Análisis de crecimiento de funciones

Jerarquía asintótica fundamental

- Teorema 1: dominancia exponencial sobre polinómica

- Teorema 2: dominancia polinómica sobre logarítmica

- Teorema 3: jerarquía asintótica completa

Análisis de complejidad en algoritmos clásicos

Análisis de recurrencias

Contenido II

Ecuación característica

Caso 1: Recurrencias homogéneas

Ejemplo: sucesión de Fibonacci (algoritmo recursivo)

Caso 2: Recurrencias lineales no homogéneas

Ejemplo: Torres de Hanoi (algoritmo recursivo)

Ejemplo completo: recurrencia no homogénea

Alcances, existencia y limitaciones

Teorema maestro

Teorema maestro: caso 1 (dominancia de las hojas)

Teorema maestro: caso 2 (equilibrio de costes)

Teorema maestro: caso 3 (dominancia de la raíz)

Árbol de recursión: fundamentos conceptuales

Ejemplos: Teorema maestro

Ejemplos clásicos

Integración teoría-práctica en el análisis de algoritmos

Contenido III

Clases fundamentales

- Clases fundamentales: definición de P

- Clases fundamentales: definición de NP

- Clases fundamentales: ejemplos representativos

Ejercicios propuestos

Ejercicios de recurrencias lineales

Aplicación del teorema maestro

Bibliografía recomendada

Fundamentos del análisis de algoritmos

Notaciones asintóticas

Notación O (cota superior)

Notación O (cota superior): fundamentos teóricos I

Definición formal

Sea $f: \mathbb{N} \rightarrow \mathbb{R}$ una función. Decimos que $f(n) \in O(g(n))$ si existen constantes reales $c > 0$ y $n_0 \in \mathbb{N}$ tales que

$$\forall n \geq n_0, \quad |f(n)| \leq c |g(n)|.$$

Interpretación algorítmica

La notación O establece una *cota superior asintótica* para el crecimiento de $f(n)$, caracterizando el comportamiento en el *peor caso* de un algoritmo cuando el tamaño de la entrada tiende a infinito.

Notación O (cota superior): fundamentos teóricos II

Criterio mediante límites

$$\lim_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} \in [0, +\infty) \implies f(n) = O(g(n)).$$

Notación O : Ejemplo 1 (enunciado) I

Determine una cota asintótica superior para la función polinómica

$$f(n) = 3n^2 + 5n + 7.$$

Notación O : Ejemplo 1 (solución) I

Para $n \geq 1$ se verifican las desigualdades elementales $n \leq n^2$ y $1 \leq n^2$. Por lo tanto,

$$f(n) = 3n^2 + 5n + 7 \leq 3n^2 + 5n^2 + 7n^2 = 15n^2.$$

Eligiendo las constantes $c = 15$ y $n_0 = 1$, se satisface la definición formal de la notación O , concluyendo que

$$f(n) = O(n^2).$$

Notación O : Ejemplo 1 (análisis Θ) I

Sea la función cociente $\phi(n) = \frac{f(n)}{n^2}$, continua, estrictamente positiva y decreciente para todo $n \geq 1$. Entonces:

$$\phi(1) = \frac{f(1)}{1^2} = 15, \quad \lim_{n \rightarrow \infty} \phi(n) = \lim_{n \rightarrow \infty} \left(3 + \frac{5}{n} + \frac{7}{n^2} \right) = 3.$$

Dado que ϕ es decreciente en $[1, \infty)$, se cumple:

$$3 < \phi(n) \leq 15, \quad \forall n \geq 1.$$

Escogiendo, por ejemplo, $c_1 = 3$ y $c_2 = 15$, obtenemos:

$$c_1 \leq \frac{f(n)}{n^2} \leq c_2, \quad \forall n \geq 1.$$

Multiplicando por $n^2 > 0$, se deduce:

$$c_1 n^2 \leq f(n) \leq c_2 n^2, \quad \forall n \geq 1,$$

lo cual implica rigurosamente que $f(n) = \Theta(n^2)$.

Notación O : Ejemplo 2 (enunciado) I

Determine una cota asintótica superior para la función logarítmica

$$f(n) = 10 \log n + 25.$$

Notación O : Ejemplo 2 (solución) I

Para $n \geq 3 > e$ se cumple $\log n \geq 1$. En consecuencia,

$$f(n) = 10 \log n + 25 \leq 10 \log n + 25 \log n = 35 \log n.$$

Con las constantes $c = 35$ y $n_0 = 3$, se satisface la definición formal, obteniéndose

$$f(n) = O(\log n).$$

Notación O : Ejemplo 2 (análisis Θ) I

La función cociente $\phi(n) = \frac{f(n)}{\log n}$ es continua, estrictamente positiva y decreciente para todo $n \geq 3$. Entonces:

$$\phi(3) = 10 + \frac{25}{\log 3}, \quad \lim_{n \rightarrow \infty} \phi(n) = \lim_{n \rightarrow \infty} \left(10 + \frac{25}{\log n} \right) = 10.$$

Dado que ϕ es decreciente en $[3, \infty)$, se cumple:

$$10 < \phi(n) \leq 10 + \frac{25}{\log 3}, \quad \forall n \geq 3.$$

Por lo tanto, tomando por ejemplo $c_1 = 10$ y $c_2 = 10 + \frac{25}{\log 3}$, obtenemos:

$$c_1 \leq \frac{f(n)}{\log n} \leq c_2, \quad \forall n \geq 3.$$

Notación O : Ejemplo 2 (análisis Θ) II

Multiplicando por $\log n > 0$ (válido para $n \geq 3$), se deduce:

$$c_1 \log n \leq f(n) \leq c_2 \log n, \quad \forall n \geq 3,$$

lo cual implica rigurosamente que $f(n) = \Theta(\log n)$.

Notación O : Ejemplo 3 (enunciado) I

Determine una cota asintótica superior para la función

$$f(n) = 7n \log n + 20n.$$

Notación O : Ejemplo 3 (solución) I

Para $n \geq 3 > e$ se verifica $\log n \geq 1$. Por lo tanto,

$$f(n) = 7n \log n + 20n \leq 7n \log n + 20n \log n = 27n \log n.$$

Eligiendo $c = 27$ y $n_0 = 3$, se concluye que

$$f(n) = O(n \log n).$$

Notación O : Ejemplo 3 (análisis Θ) I

La función cociente $\phi(n) = \frac{f(n)}{n \log n}$ es continua, estrictamente positiva y decreciente para todo $n \geq 3$. En efecto,

$$\phi(n) = 7 + \frac{20}{\log n},$$

y como $\log n$ es creciente y positivo en $[3, \infty)$, el término $\frac{20}{\log n}$ es decreciente, luego ϕ también lo es.

Se tiene:

$$\phi(3) = 7 + \frac{20}{\log 3}, \quad \lim_{n \rightarrow \infty} \phi(n) = \lim_{n \rightarrow \infty} \left(7 + \frac{20}{\log n} \right) = 7.$$

Dado que ϕ es decreciente en $[3, \infty)$, se cumple:

$$7 < \phi(n) \leq 7 + \frac{20}{\log 3}, \quad \forall n \geq 3.$$

Notación O : Ejemplo 3 (análisis Θ) II

Por tanto, tomando $c_1 = 7$ y $c_2 = 7 + \frac{20}{\log 3}$, obtenemos:

$$c_1 \leq \frac{f(n)}{n \log n} \leq c_2, \quad \forall n \geq 3.$$

Como $n \log n > 0$ para $n \geq 3$, al multiplicar por esta cantidad se deduce:

$$c_1 n \log n \leq f(n) \leq c_2 n \log n, \quad \forall n \geq 3,$$

lo cual implica rigurosamente que $f(n) = \Theta(n \log n)$.

Notación Ω (cota inferior)

Notación Ω (cota inferior): fundamentos teóricos I

Definición formal

Sea $f: \mathbb{N} \rightarrow \mathbb{R}$ una función. Decimos que $f(n) \in \Omega(g(n))$ si existen constantes reales $c > 0$ y $n_0 \in \mathbb{N}$ tales que

$$\forall n \geq n_0, \quad |f(n)| \geq c |g(n)|.$$

Interpretación algorítmica

La notación Ω establece una *cota inferior asintótica* para el crecimiento de $f(n)$, caracterizando el comportamiento en el *mejor caso* de un algoritmo cuando $n \rightarrow \infty$.

Notación Ω (cota inferior): fundamentos teóricos II

Criterio mediante límites

$$\lim_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} \in (0, +\infty] \implies f(n) = \Omega(g(n)).$$

Notación Ω : Ejemplo 1 (enunciado) I

Determine una cota asintótica inferior para la función lineal

$$f(n) = 2n + 4.$$

Notación Ω : Ejemplo 1 (solución) I

Para $n \geq 1$ se cumple $4 \geq 0$, de donde

$$f(n) = 2n + 4 \geq 2n.$$

Tomando $c = 2$ y $n_0 = 1$, se satisface la definición formal, concluyendo que

$$f(n) = \Omega(n).$$

Notación Ω : Ejemplo 1 I

La función cociente $\phi(n) = \frac{f(n)}{n}$ es continua, estrictamente positiva y decreciente para todo $n \geq 1$. En efecto,

$$\phi(n) = 2 + \frac{4}{n},$$

y como $\frac{4}{n}$ es decreciente en $[1, \infty)$, también lo es ϕ .

Se calculan:

$$\phi(1) = 2 + \frac{4}{1} = 6, \quad \lim_{n \rightarrow \infty} \phi(n) = \lim_{n \rightarrow \infty} \left(2 + \frac{4}{n} \right) = 2.$$

Dado que ϕ es decreciente en $[1, \infty)$, se cumple:

$$2 < \phi(n) \leq 6, \quad \forall n \geq 1.$$

Notación Ω : Ejemplo 1 II

Por tanto, tomando $c_1 = 2$ y $c_2 = 6$, obtenemos:

$$c_1 \leq \frac{f(n)}{n} \leq c_2, \quad \forall n \geq 1.$$

Como $n > 0$ para $n \geq 1$, al multiplicar por n se deduce:

$$c_1 n \leq f(n) \leq c_2 n, \quad \forall n \geq 1,$$

lo cual implica rigurosamente que $f(n) = \Theta(n)$.

Notación Ω : Ejemplo 2 (enunciado) I

Determine una cota asintótica inferior para la función cuadrática

$$f(n) = 4n^2 + 3n + 10.$$

Notación Ω : Ejemplo 2 (solución) I

Para $n \geq 1$ se cumple $n^2 \geq n$ y $n^2 \geq 1$. Por lo tanto,

$$f(n) = 4n^2 + 3n + 10 \geq 4n^2.$$

Con $c = 4$ y $n_0 = 1$, se obtiene

$$f(n) = \Omega(n^2).$$

Notación Ω : Ejemplo 2 (análisis Θ) I

La función cociente $\phi(n) = \frac{f(n)}{n^2}$ es continua, estrictamente positiva y decreciente para todo $n \geq 1$. En efecto,

$$\phi(n) = 4 + \frac{3}{n} + \frac{10}{n^2},$$

y como cada término fraccionario es decreciente en $[1, \infty)$, también lo es ϕ . Se tiene:

$$\phi(1) = 4 + \frac{3}{1} + \frac{10}{1^2} = 17, \quad \lim_{n \rightarrow \infty} \phi(n) = \lim_{n \rightarrow \infty} \left(4 + \frac{3}{n} + \frac{10}{n^2} \right) = 4.$$

Dado que ϕ es decreciente en $[1, \infty)$, se cumple:

$$4 < \phi(n) \leq 17, \quad \forall n \geq 1.$$

Notación Ω : Ejemplo 2 (análisis Θ) II

Por tanto, tomando $c_1 = 4$ y $c_2 = 17$, obtenemos:

$$c_1 \leq \frac{f(n)}{n^2} \leq c_2, \quad \forall n \geq 1.$$

Como $n^2 > 0$ para $n \geq 1$, al multiplicar por n^2 se deduce:

$$c_1 n^2 \leq f(n) \leq c_2 n^2, \quad \forall n \geq 1,$$

lo cual implica rigurosamente que $f(n) = \Theta(n^2)$.

Notación Ω : Ejemplo 3 (enunciado) I

Determine una cota asintótica inferior para la función

$$f(n) = 5 \log n + 12.$$

Notación Ω : Ejemplo 3 (solución) I

Para $n \geq 2$ se verifica $\log n \geq 1$, de donde

$$f(n) = 5 \log n + 12 \geq 5 \log n.$$

Con $c = 5$ y $n_0 = 2$, se concluye que

$$f(n) = \Omega(\log n).$$

Notación Ω : Ejemplo 3 (análisis Θ) I

La función cociente $\phi(n) = \frac{f(n)}{\log n}$ es continua, estrictamente positiva y decreciente para todo $n \geq 2$. En efecto,

$$\phi(n) = 5 + \frac{12}{\log n},$$

y como $\log n$ es creciente y positivo en $[2, \infty)$, el término $\frac{12}{\log n}$ es decreciente, luego ϕ también lo es.

Se tiene:

$$\phi(2) = 5 + \frac{12}{\log 2}, \quad \lim_{n \rightarrow \infty} \phi(n) = \lim_{n \rightarrow \infty} \left(5 + \frac{12}{\log n} \right) = 5.$$

Dado que ϕ es decreciente en $[2, \infty)$, se cumple:

$$5 < \phi(n) \leq 5 + \frac{12}{\log 2}, \quad \forall n \geq 2.$$

Notación Ω : Ejemplo 3 (análisis Θ) II

Por tanto, tomando $c_1 = 5$ y $c_2 = 5 + \frac{12}{\log 2}$, obtenemos:

$$c_1 \leq \frac{f(n)}{\log n} \leq c_2, \quad \forall n \geq 2.$$

Como $\log n > 0$ para $n \geq 2$, al multiplicar por $\log n$ se deduce:

$$c_1 \log n \leq f(n) \leq c_2 \log n, \quad \forall n \geq 2,$$

lo cual implica rigurosamente que $f(n) = \Theta(\log n)$.

Notación Θ (cota ajustada)

Notación Θ (cota ajustada): fundamentos teóricos I

Definición formal

Sea $f: \mathbb{N} \rightarrow \mathbb{R}$ una función. Decimos que $f(n) \in \Theta(g(n))$ si existen constantes reales $c_1, c_2 > 0$ y $n_0 \in \mathbb{N}$ tales que

$$\forall n \geq n_0, \quad c_1 |g(n)| \leq |f(n)| \leq c_2 |g(n)|.$$

Interpretación algorítmica

La notación Θ establece una *cota ajustada asintótica* para el crecimiento de $f(n)$, es decir, describe funciones cuyo orden de crecimiento está acotado tanto superior como inferiormente por $g(n)$.

Notación Θ (cota ajustada): fundamentos teóricos II

Criterio mediante límites

$$\lim_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} = c \in (0, +\infty) \quad \implies \quad f(n) = \Theta(g(n)).$$

Notación Θ : Ejemplo 1 (enunciado) I

Demuestre que la función lineal

$$f(n) = 4n + 10$$

pertenece a $\Theta(n)$.

Notación Θ : Ejemplo 1 (solución) I

Para $n \geq 1$ se verifican las desigualdades

$$4n \leq 4n + 10 \leq 4n + 10n = 14n.$$

Tomando $c_1 = 4$, $c_2 = 14$ y $n_0 = 1$, se satisfacen simultáneamente las cotas inferior y superior requeridas por la definición, concluyendo que

$$f(n) = \Theta(n).$$

Notación Θ : Ejemplo 1 (análisis mediante límites) I

La función cociente $\phi(n) = \frac{f(n)}{n}$ es continua, estrictamente positiva y decreciente para todo $n \geq 1$. En efecto,

$$\phi(n) = 4 + \frac{10}{n},$$

y como $\frac{10}{n}$ es decreciente en $[1, \infty)$, también lo es ϕ .

Se tiene:

$$\phi(1) = 4 + \frac{10}{1} = 14, \quad \lim_{n \rightarrow \infty} \phi(n) = \lim_{n \rightarrow \infty} \left(4 + \frac{10}{n}\right) = 4.$$

Dado que ϕ es decreciente en $[1, \infty)$, se cumple:

$$4 < \phi(n) \leq 14, \quad \forall n \geq 1.$$

Notación Θ : Ejemplo 1 (análisis mediante límites) II

Por tanto, tomando $c_1 = 4$ y $c_2 = 14$, obtenemos:

$$c_1 \leq \frac{f(n)}{n} \leq c_2, \quad \forall n \geq 1.$$

Como $n > 0$ para $n \geq 1$, al multiplicar por n se deduce:

$$c_1 n \leq f(n) \leq c_2 n, \quad \forall n \geq 1,$$

lo cual confirma rigurosamente que $f(n) = \Theta(n)$.

Notación Θ : Ejemplo 2 (enunciado) I

Demuestre que la función cuadrática

$$f(n) = 5n^2 + 3n + 8$$

pertenece a $\Theta(n^2)$.

Notación Θ : Ejemplo 2 (solución) I

Para $n \geq 1$ se cumplen $n \leq n^2$ y $1 \leq n^2$, de donde

$$5n^2 \leq 5n^2 + 3n + 8 \leq 5n^2 + 3n^2 + 8n^2 = 16n^2.$$

Con $c_1 = 5$, $c_2 = 16$ y $n_0 = 1$, se satisfacen las cotas requeridas, obteniéndose

$$f(n) = \Theta(n^2).$$

Notación Θ : Ejemplo 2 (análisis mediante límites) I

La función cociente $\phi(n) = \frac{f(n)}{n^2}$ es continua, estrictamente positiva y decreciente para todo $n \geq 1$. En efecto,

$$\phi(n) = 5 + \frac{3}{n} + \frac{8}{n^2},$$

y como los términos $\frac{3}{n}$ y $\frac{8}{n^2}$ son decrecientes en $[1, \infty)$, también lo es ϕ . Se tiene:

$$\phi(1) = 5 + \frac{3}{1} + \frac{8}{1^2} = 16, \quad \lim_{n \rightarrow \infty} \phi(n) = \lim_{n \rightarrow \infty} \left(5 + \frac{3}{n} + \frac{8}{n^2} \right) = 5.$$

Dado que ϕ es decreciente en $[1, \infty)$, se cumple:

$$5 < \phi(n) \leq 16, \quad \forall n \geq 1.$$

Notación Θ : Ejemplo 2 (análisis mediante límites) II

Por tanto, tomando $c_1 = 5$ y $c_2 = 16$, obtenemos:

$$c_1 \leq \frac{f(n)}{n^2} \leq c_2, \quad \forall n \geq 1.$$

Como $n^2 > 0$ para $n \geq 1$, al multiplicar por n^2 se deduce:

$$c_1 n^2 \leq f(n) \leq c_2 n^2, \quad \forall n \geq 1,$$

lo cual confirma rigurosamente que $f(n) = \Theta(n^2)$.

Notación Θ : Ejemplo 3 (enunciado) I

Demuestre que la función

$$f(n) = 6n \log n + 20n$$

pertenece a $\Theta(n \log n)$.

Notación Θ : Ejemplo 3 (solución) I

Para $n \geq e$ se cumple $\log n \geq 1$, por lo que

$$6n \log n \leq 6n \log n + 20n \leq 6n \log n + 20n \log n = 26n \log n.$$

Eligiendo $c_1 = 6$, $c_2 = 26$ y $n_0 = \lceil e \rceil = 3$, se concluye que

$$f(n) = \Theta(n \log n).$$

Notación Θ : Ejemplo 3 (análisis mediante límites) I

La función cociente $\phi(n) = \frac{f(n)}{n \log n}$ es continua, estrictamente positiva y decreciente para todo $n \geq 3$. En efecto,

$$\phi(n) = 6 + \frac{20}{\log n},$$

y como $\log n$ es creciente y positivo en $[3, \infty)$, el término $\frac{20}{\log n}$ es decreciente, luego ϕ también lo es.

Se tiene:

$$\phi(3) = 6 + \frac{20}{\log 3}, \quad \lim_{n \rightarrow \infty} \phi(n) = \lim_{n \rightarrow \infty} \left(6 + \frac{20}{\log n} \right) = 6.$$

Dado que ϕ es decreciente en $[3, \infty)$, se cumple:

$$6 < \phi(n) \leq 6 + \frac{20}{\log 3}, \quad \forall n \geq 3.$$

Notación Θ : Ejemplo 3 (análisis mediante límites) II

Por tanto, tomando $c_1 = 6$ y $c_2 = 6 + \frac{20}{\log 3}$, obtenemos:

$$c_1 \leq \frac{f(n)}{n \log n} \leq c_2, \quad \forall n \geq 3.$$

Como $n \log n > 0$ para $n \geq 3$, al multiplicar por $n \log n$ se deduce:

$$c_1 n \log n \leq f(n) \leq c_2 n \log n, \quad \forall n \geq 3,$$

lo cual confirma rigurosamente que $f(n) = \Theta(n \log n)$.

Relación formal y aplicación práctica de notaciones asintóticas

Relación formal entre notaciones I

$$\Theta(g(n)) = O(g(n)) \cap \Omega(g(n)).$$

Interpretación conceptual

- ▶ $O(g(n))$: cota superior asintótica.
- ▶ $\Omega(g(n))$: cota inferior asintótica.
- ▶ $\Theta(g(n))$: cota ajustada (superior e inferior simultáneamente).

Relación formal entre notaciones II

Interpretación multiplicativa

Formulación que resalta la naturaleza de factor de escala:

$$O(g(n)) = g(n) \cdot O(1), \quad \Omega(g(n)) = g(n) \cdot \Omega(1), \quad \Theta(g(n)) = g(n) \cdot \Theta(1).$$

Estas igualdades expresan que las funciones en cada clase crecen como $g(n)$ multiplicado por una función acotada (superior, inferior o ajustadamente).

Relación formal entre notaciones III

Errores conceptuales frecuentes

- ▶ Interpretar $O(n)$ como tiempo exacto (es una cota, no igualdad).
- ▶ Descartar $O(n^2)$ sin considerar contexto o tamaño realista de entrada.
- ▶ Ignorar que constantes absorbidas pueden ser relevantes para n moderado.
- ▶ Confundir mejor caso (Ω), peor caso (O) y caso promedio.

Complementariedad teoría-experimentación

El análisis asintótico teórico requiere validación empírica mediante mediciones de tiempo y simulaciones para una caracterización completa del comportamiento algorítmico.

Integración metodológica II

Herramientas de visualización

- ▶ GEOGEBRA: exploración interactiva inicial
- ▶ PYTHON/MATPLOTLIB: generación de gráficos a gran escala
- ▶ SAGEMATH/MAXIMA: cálculo simbólico y numérico integrado

Práctica recomendada

Comparación gráfica de órdenes de crecimiento fundamentales:

$$n, n \log n, n^2, 2^n, n!$$

para desarrollar intuición sobre diferencias asintóticas.

Fundamentos de la notación asintótica

Fundamentos de la notación asintótica I

Motivación teórica

La notación asintótica proporciona un marco formal para el análisis de algoritmos mediante:

- ▶ La abstracción de constantes multiplicativas y términos de orden inferior.
- ▶ El estudio del comportamiento límite cuando el tamaño de la entrada n tiende a infinito.

Objetivos metodológicos

- ▶ Caracterizar la eficiencia teórica de algoritmos independientemente de la plataforma de ejecución.
- ▶ Establecer criterios rigurosos para la clasificación y comparación de algoritmos según su escalabilidad.

Ejemplo: cota asintótica de función cuadrática (enunciado) I

Considérese un algoritmo cuyo tiempo de ejecución está dado por

$$T(n) = 3n^2 + 2n + 7.$$

Determine una cota asintótica superior para $T(n)$ utilizando la definición formal de la notación O .

Ejemplo: cota asintótica de función cuadrática (solución) I

Para $n \geq 1$ se verifican las desigualdades elementales $n \leq n^2$ y $1 \leq n^2$. En consecuencia,

$$2n \leq 2n^2.$$

Además,

$$7 \leq 7n^2.$$

Por lo tanto,

$$T(n) = 3n^2 + 2n + 7 \leq 3n^2 + 2n^2 + 7n^2 = 12n^2.$$

Eligiendo $c = 12$ y $n_0 = 1$, se satisface la definición formal de la notación O , concluyendo que

$$T(n) = O(n^2).$$

Esta expresión indica que el algoritmo presenta complejidad cuadrática en el peor caso.

Clasificación de casos algorítmicos I

Tipología según comportamiento de entrada

- ▶ **Mejor caso (Ω):** escenario más favorable, correspondiente a instancias que minimizan el consumo de recursos computacionales.
- ▶ **Caso promedio:** comportamiento esperado bajo una distribución probabilística sobre las instancias; frecuentemente se asume distribución uniforme.
- ▶ **Peor caso (O):** escenario más desfavorable, correspondiente a instancias que maximizan el consumo de recursos; constituye la medida predominante en análisis teórico por su carácter conservador.

Ejemplo: análisis de búsqueda lineal (enunciado) I

Analice el algoritmo de búsqueda lineal en un arreglo no ordenado de n elementos. Determine su complejidad asintótica en los tres escenarios fundamentales (mejor caso, caso promedio y peor caso), expresando cada resultado mediante la notación asintótica apropiada y justificando rigurosamente las cotas obtenidas.

Ejemplo: análisis de búsqueda lineal (solución) I

Mejor caso. El elemento buscado se encuentra en la primera posición del arreglo. El número de comparaciones es constante:

$$T_{\text{mejor}}(n) = 1.$$

Por lo tanto,

$$T_{\text{mejor}}(n) = \Theta(1).$$

Peor caso. El elemento no pertenece al arreglo o se encuentra en la última posición. Se requieren exactamente n comparaciones:

$$T_{\text{peor}}(n) = n.$$

Por lo tanto,

$$T_{\text{peor}}(n) = \Theta(n).$$

Ejemplo: análisis de búsqueda lineal (solución) II

Caso promedio. Asumiendo distribución uniforme sobre las $n+1$ posibilidades (las n posiciones del arreglo más la ausencia del elemento), el número esperado de comparaciones es

$$T_{\text{prom}}(n) = \frac{1}{n+1} \sum_{k=1}^n k + \frac{n}{n+1} = \frac{n+2}{2}.$$

Para $n \geq 1$ se verifica

$$\frac{1}{2}n \leq \frac{n+2}{2} \leq n.$$

Por lo tanto,

$$T_{\text{prom}}(n) = \Theta(n).$$

Observamos que, pese a diferencias en las constantes, tanto el caso promedio como el peor caso presentan complejidad lineal asintótica.

Jerarquía de órdenes de crecimiento I

Definición de notación o (o chico)

Sea $f: \mathbb{N} \rightarrow \mathbb{R}$ una función. Decimos que $f(n) \in o(g(n))$ si para toda constante $c > 0$ existe $n_0 \in \mathbb{N}$ tal que

$$\forall n \geq n_0, \quad |f(n)| \leq c |g(n)|.$$

Equivalentemente,

$$\lim_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} = 0 \quad \Longleftrightarrow \quad f(n) = o(g(n)).$$

Jerarquía de órdenes de crecimiento II

Relación entre o y O

Se verifica la inclusión estricta $o(g(n)) \subsetneq O(g(n))$. Es decir:

$$f(n) = o(g(n)) \implies f(n) = O(g(n)),$$

pero el recíproco no es cierto en general. La diferencia esencial radica en que O permite una constante fija $c > 0$ que acota a $f(n)$, mientras que o exige que dicha cota valga para *toda* constante positiva, lo que implica un crecimiento estrictamente menor.

Existe un orden estricto de crecimiento entre las funciones prototípicas fundamentales en análisis de algoritmos:

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!.$$

Jerarquía de órdenes de crecimiento III

Dicho orden se interpreta formalmente como: para cada par consecutivo $f(n)$, $g(n)$ en la secuencia,

$$\lim_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} = 0,$$

lo cual establece $f(n) = o(g(n))$.

Ejemplo: verificación de dominancia asintótica (enunciado) I

Verifique formalmente mediante el cálculo de límites que

$$n \log n = o(n^2),$$

es decir, demuestre que $n \log n$ crece estrictamente más lentamente que n^2 cuando $n \rightarrow \infty$.

Ejemplo: verificación de dominancia asintótica (solución) I

Consideremos el límite del cociente entre ambas funciones:

$$\lim_{n \rightarrow \infty} \frac{|n \log n|}{|n^2|} = \lim_{n \rightarrow \infty} \frac{\log n}{n}.$$

Aplicando la regla de L'Hôpital (dado que el límite presenta la forma indeterminada ∞/∞),

$$\lim_{n \rightarrow \infty} \frac{\log n}{n} = \lim_{n \rightarrow \infty} \frac{1/n}{1} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0.$$

Por lo tanto,

$$\lim_{n \rightarrow \infty} \frac{|n \log n|}{|n^2|} = 0.$$

Lo cual establece rigurosamente que $n \log n = o(n^2)$ y, en consecuencia, $n \log n = O(n^2)$ pero $n^2 \notin O(n \log n)$.

Valores umbral de dominancia I

Valores mínimos de n_0 para observar la dominancia empírica entre funciones consecutivas:

- ▶ $\log_2 n < n$ para todo $n \geq 2$.
- ▶ $n < n \log_2 n$ para todo $n \geq 3$.
- ▶ $n \log_2 n < n^2$ para todo $n \geq 2$.
- ▶ $n^2 < n^3$ para todo $n \geq 2$.
- ▶ $n^3 < 2^n$ para todo $n \geq 10$.
- ▶ $2^n < n!$ para todo $n \geq 4$.

Estos umbrales ilustran que, aunque el orden asintótico está determinado por el comportamiento cuando $n \rightarrow \infty$, en la práctica las transiciones de dominancia pueden requerir valores relativamente grandes de n .

Implicaciones prácticas del crecimiento asintótico I

Principios fundamentales

- ▶ Los algoritmos cuyas funciones de complejidad presentan menor tasa de crecimiento asintótico escalan significativamente mejor ante incrementos en el tamaño de la entrada.
- ▶ Diferencias aparentemente menores en la expresión analítica pueden traducirse en disparidades computacionales abismales para instancias de tamaño realista.

Ejemplo numérico: enunciado I

Compare el número de operaciones básicas requeridas por algoritmos con complejidades $n \log_2 n$, n^2 y 2^n cuando el tamaño de entrada es $n = 10^6$.

Ejemplo numérico: solución I

Para $n = 10^6$ se obtienen los siguientes valores:

$$n \log_2 n = 10^6 \cdot \log_2(10^6) \approx 10^6 \cdot 19.93 \approx 2 \cdot 10^7.$$

$$n^2 = (10^6)^2 = 10^{12}.$$

$$2^n = 2^{10^6}.$$

El valor 2^{10^6} es computacionalmente intratable; su representación decimal requiere aproximadamente $\log_{10}(2^{10^6}) \approx 301\,030$ dígitos.

Observamos que n^2 supera a $n \log_2 n$ en cinco órdenes de magnitud, mientras que 2^n es incomparablemente mayor que ambas funciones polinómicas.

Tiempo vs espacio

Análisis de complejidad temporal I

Definición

La complejidad temporal de un algoritmo se define como el número de operaciones elementales ejecutadas en función del tamaño n de la entrada.

Operaciones elementales

Incluyen comparaciones, asignaciones, accesos a memoria y operaciones aritméticas básicas, consideradas de costo unitario en el modelo RAM (Random Access Machine).

Análisis de complejidad temporal II

Clasificación asintótica

El crecimiento del número de operaciones se caracteriza mediante notación asintótica:

$$O(1), O(\log n), O(n), O(n \log n), O(n^2), O(2^n), O(n!).$$

Análisis de complejidad espacial I

Definición

La complejidad espacial de un algoritmo se define como la cantidad total de memoria requerida durante su ejecución en función del tamaño n de la entrada.

Componentes de la memoria

- ▶ Memoria para almacenar la entrada y la salida.
- ▶ Memoria auxiliar: variables temporales, estructuras de datos adicionales y pila de recursión.
- ▶ Memoria fija independiente del tamaño de entrada (constantes, código del programa).

Análisis de complejidad espacial II

Clasificación asintótica

El consumo de memoria se caracteriza mediante notación asintótica:

$$O(1), O(\log n), O(n), O(n^2).$$

Relación tiempo-espacio y criterios de selección I

Trade-off fundamental

Existe frecuentemente una relación de compromiso (trade-off) entre eficiencia temporal y espacial. Algoritmos que optimizan tiempo pueden requerir mayor memoria, y viceversa.

Criterios para la selección de algoritmos

- ▶ Restricciones del entorno de ejecución (memoria limitada en dispositivos embebidos).
- ▶ Requisitos de tiempo real o latencia crítica.
- ▶ Tamaño típico de las instancias del problema en la práctica.
- ▶ Disponibilidad de recursos computacionales (CPU vs memoria).

Relación tiempo-espacio y criterios de selección II

Principio metodológico

El análisis teórico debe complementarse con mediciones empíricas en escenarios representativos para tomar decisiones informadas sobre el equilibrio tiempo-espacio.

Ejemplo: suma de los primeros n naturales (enunciado) I

Problema. Calcular la suma de los primeros n números naturales:

$$S(n) = \sum_{i=1}^n i.$$

Determine la complejidad temporal y espacial del algoritmo iterativo que resuelve este problema utilizando el modelo RAM.

Ejemplo: suma de los primeros n naturales (algoritmo) I

Algoritmo 1 Sumalterativa(n)

```
1: function SUMAITERATIVA( $n$ )  
2:    $suma \leftarrow 0$   
3:   for  $i = 1$  to  $n$  do  
4:      $suma \leftarrow suma + i$   
5:   end for  
6:   return  $suma$   
7: end function
```

Ejemplo: suma de los primeros n naturales (análisis de operaciones) I

Conteo detallado de operaciones elementales:

Línea	Operación	Tipo	Costo	Frecuencia
2	$suma \leftarrow 0$	Asignación	1	1
3	$i \leftarrow 1$	Asignación	1	1
3	$i \leq n$	Comparación	1	$n + 1$
4	$suma + i$	Suma	1	n
4	$suma \leftarrow \dots$	Asignación	1	n
3	$i + 1$	Suma	1	n
3	$i \leftarrow \dots$	Asignación	1	n
6	return $suma$	Retorno	1	1

Ejemplo: suma de los primeros n naturales (costo temporal)

|

El costo temporal total se obtiene sumando las contribuciones:

$T(n) = 1 \cdot 1$	(línea 2)
$+ 1 \cdot 1$	(inicialización i)
$+ 1 \cdot (n + 1)$	(comparaciones bucle)
$+ 1 \cdot n$	(sumas en línea 4)
$+ 1 \cdot n$	(asignaciones en línea 4)
$+ 1 \cdot n$	(sumas en incremento)
$+ 1 \cdot n$	(asignaciones en incremento)
$+ 1 \cdot 1$	(retorno)
$= 1 + 1 + n + 1 + n + n + n + n + 1$	
$= 5n + 4.$	

Ejemplo: suma de los primeros n naturales (costo temporal)

II

Para $n \geq 1$:

$$5n + 4 \leq 5n + 4n = 9n.$$

Por lo tanto, con $c = 9$ y $n_0 = 1$:

$$T(n) = O(n).$$

Ejemplo: suma de los primeros n naturales (costo espacial) I

Análisis de consumo de memoria

- ▶ Variable n : entrada del algoritmo (1 unidad).
- ▶ Variable $suma$: acumulador (1 unidad).
- ▶ Variable i : contador del bucle (1 unidad).
- ▶ Código del algoritmo: memoria constante.

El consumo total es independiente de n :

$$M(n) = 3 + \text{constante} = O(1).$$

Ejemplo: suma de los primeros n naturales (conclusión) I

Métrica	Expresión exacta	Complejidad
Tiempo	$5n + 4$	$O(n)$
Espacio	constante	$O(1)$

Observación

La fórmula cerrada $S(n) = n(n+1)/2$ permite resolver el problema en tiempo $O(1)$, ilustrando cómo el conocimiento matemático puede transformar radicalmente la eficiencia algorítmica.

Ejemplo: ordenamiento burbuja (enunciado) I

Problema. Ordenar un arreglo $A[0 \dots n - 1]$ de n elementos en orden no decreciente mediante comparaciones e intercambios sucesivos de elementos adyacentes.

Determine la complejidad temporal y espacial del algoritmo de ordenamiento burbuja utilizando el modelo RAM.

Ejemplo: ordenamiento burbuja (algoritmo) I

Algoritmo 2 Burbuja(A, n)

```
1: function BURBUJA( $A, n$ )
2:   for  $i = 0$  to  $n - 2$  do
3:     for  $j = 0$  to  $n - i - 2$  do
4:       if  $A[j] > A[j + 1]$  then
5:         Intercambiar  $A[j]$  y  $A[j + 1]$ 
6:       end if
7:     end for
8:   end for
9:   return  $A$ 
10: end function
```

Ejemplo: ordenamiento burbuja (análisis de operaciones) I

Conteo detallado de operaciones elementales:

Línea	Operación	Tipo	Costo	Frecuencia
2	$i \leftarrow 0$	Asignación	1	1
2	$i \leq n - 2$	Comparación	1	n
3	$j \leftarrow 0$	Asignación	1	$n - 1$
3	$j \leq n - i - 2$	Comparación	1	$\sum_{i=0}^{n-2} (n - i - 1) + 1$
4	$A[j] > A[j + 1]$	Comparación	1	$\sum_{i=0}^{n-2} (n - i - 1)$
5	$t \leftarrow A[j]$	Asignación	1	$\leq \sum_{i=0}^{n-2} (n - i - 1)$

Ejemplo: ordenamiento burbuja (análisis de operaciones) II

Línea	Operación	Tipo	Costo	Frecuencia
5	$A[j] \leftarrow A[j + 1]$	Asignación	1	$\leq \sum_{i=0}^{n-2} (n - i - 1)$
5	$A[j + 1] \leftarrow t$	Asignación	1	$\leq \sum_{i=0}^{n-2} (n - i - 1)$
3	$j \leftarrow j + 1$	Asignación+suma	2	$\sum_{i=0}^{n-2} (n - i - 1)$
2	$i \leftarrow i + 1$	Asignación+suma	2	$n - 1$
9	return A	Retorno	1	1

Ejemplo: ordenamiento burbuja (costo temporal) I

Definimos la sumatoria auxiliar:

$$S = \sum_{i=0}^{n-2} (n - i - 1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}.$$

Ejemplo: ordenamiento burbuja (costo temporal) II

El costo temporal total es:

$T(n) = 1 \cdot 1$	(inicialización i)
$+ 1 \cdot n$	(comparaciones bucle externo)
$+ 1 \cdot (n - 1)$	(inicializaciones j)
$+ 1 \cdot (S + 1)$	(comparaciones bucle interno)
$+ 1 \cdot S$	(comparaciones elementos)
$+ 3 \cdot S$	(intercambios: 3 asignaciones)
$+ 2 \cdot S$	(incremento j)
$+ 2 \cdot (n - 1)$	(incremento i)
$+ 1 \cdot 1$	(retorno)
$= 1 + n + n - 1 + S + 1 + S +$	
$+ 3S + 2S + 2n - 2 + 1$	
$= 7S + 4n.$	

Ejemplo: ordenamiento burbuja (costo temporal) III

Sustituyendo $S = \frac{n(n-1)}{2}$:

$$T(n) = 7 \cdot \frac{n(n-1)}{2} + 4n = \frac{7n^2 - 7n + 8n}{2} = \frac{7n^2 + n}{2}.$$

Para $n \geq 1$:

$$\frac{7n^2 + n}{2} \leq \frac{7n^2 + n^2}{2} = 4n^2.$$

Por lo tanto, con $c = 4$ y $n_0 = 1$:

$$T(n) = O(n^2).$$

Ejemplo: ordenamiento burbuja (costo espacial) I

Análisis de consumo de memoria

- ▶ Arreglo $A[0 \dots n - 1]$: n posiciones (entrada).
- ▶ Variable n : tamaño del arreglo (1 unidad).
- ▶ Variable i : índice bucle externo (1 unidad).
- ▶ Variable j : índice bucle interno (1 unidad).
- ▶ Variable t : temporal para intercambio (1 unidad).
- ▶ Código del algoritmo: memoria constante.

El algoritmo opera *in situ*. El consumo adicional es constante:

$$M(n) = 4 + \text{constante} = O(1).$$

Ejemplo: ordenamiento burbuja (conclusión) I

Métrica	Expresión exacta	Complejidad
Tiempo	$\frac{7n^2 + n}{2}$	$O(n^2)$
Espacio	constante	$O(1)$

Observación

El ordenamiento burbuja presenta complejidad cuadrática en tiempo pero constante en espacio. Algoritmos como mergesort ($O(n \log n)$ tiempo, $O(n)$ espacio) o quicksort ($O(n \log n)$ promedio, $O(\log n)$ espacio) ofrecen mejor escalabilidad temporal a cambio de mayor consumo espacial, ilustrando el trade-off fundamental tiempo-espacio en el diseño algorítmico.

Ejemplo: Fibonacci recursivo (enunciado) I

Problema. Calcular el n -ésimo término de la sucesión de Fibonacci definida por:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ para } n \geq 2.$$

Analice la complejidad temporal del algoritmo recursivo directo sin memoización.

Ejemplo: Fibonacci recursivo (algoritmo) I

Algoritmo 3 FibRecursivo(n)

```
1: function FIBRECURSIVO( $n$ )
2:   if  $n \leq 1$  then
3:     return  $n$ 
4:   else
5:     return FIBRECURSIVO( $n - 1$ ) + FIBRECURSIVO( $n - 2$ )
6:   end if
7: end function
```

Ejemplo: Fibonacci recursivo (análisis de operaciones) I

Relación de recurrencia para el costo temporal:

Línea	Operación	Tipo	Costo	Frecuencia
2	Comparación $n \leq 1$	Condicional	1	1 vez
3	return n	Salida	1	1 (si $n \leq 1$)
5	Llamadas recursivas	Recursión	$T(n-1) + T(n-2) + 2$	1 (si $n > 1$)

La función de costo $T(n)$ satisface la recurrencia:

$$T(n) = \begin{cases} 2 & \text{si } n \leq 1, \\ T(n-1) + T(n-2) + 2 & \text{si } n > 1. \end{cases}$$

Ejemplo: Fibonacci recursivo (solución de la recurrencia) I

Consideramos la recurrencia homogénea asociada $T_h(n) = T_h(n-1) + T_h(n-2)$ cuya ecuación característica es:

$$r^2 - r - 1 = 0.$$

Cuyas raíces son:

$$r_1 = \frac{1 + \sqrt{5}}{2}, \quad r_2 = \frac{1 - \sqrt{5}}{2}.$$

La solución general de la recurrencia no homogénea es:

$$T(n) = c_1 r_1^n + c_2 r_2^n + c_3.$$

Sustituyendo en la recurrencia original:

$$c_1 r_1^n + c_2 r_2^n + c_3 = c_1 r_1^{n-1} + c_2 r_2^{n-1} + c_3 + c_1 r_1^{n-2} + c_2 r_2^{n-2} + c_3 + 2.$$

De donde se deduce $c_3 = -2$ y las raíces satisfacen $r_i^2 - r_i - 1 = 0$ para $i = 1, 2$.

Ejemplo: Fibonacci recursivo (determinación de constantes)

Aplicando las condiciones iniciales:

$$\begin{cases} T(0) = c_1 + c_2 - 2 = 2, \\ T(1) = c_1 r_1 + c_2 r_2 - 2 = 2, \end{cases}$$

obtenemos:

$$c_1 = \frac{4r_1}{r_1 - r_2}, \quad c_2 = -\frac{4r_2}{r_1 - r_2}.$$

Por lo tanto:

$$T(n) = \frac{4}{r_1 - r_2} (r_1^{n+1} - r_2^{n+1}) - 2.$$

Como $r_1 - r_2 = \sqrt{5}$, esta expresión corresponde a cuatro veces el $(n+1)$ -ésimo número de Fibonacci menos dos.

Ejemplo: Fibonacci recursivo (cota asintótica) I

Notemos que $|r_2| < 1 < r_1$, por lo que $|r_2/r_1| < 1$. Entonces:

$$\lim_{n \rightarrow \infty} \left(\frac{r_2}{r_1} \right)^n = 0.$$

Consideremos el cociente:

$$\frac{T(n)}{r_1^n} = \frac{4r_1}{\sqrt{5}} \left(1 - \left(\frac{r_2}{r_1} \right)^{n+1} \right) - \frac{2}{r_1^n}.$$

Como $|r_2/r_1|^{n+1} \rightarrow 0$ y $2/r_1^n \rightarrow 0$ cuando $n \rightarrow \infty$, existe $n_0 \in \mathbb{N}$ y constantes $c_1, c_2 > 0$ tales que:

$$c_1 r_1^n \leq T(n) \leq c_2 r_1^n, \quad \forall n \geq n_0.$$

Ejemplo: Fibonacci recursivo (cota asintótica) II

Por lo tanto:

$$T(n) = \Theta(r_1^n) = \Theta\left(\left(\frac{1 + \sqrt{5}}{2}\right)^n\right).$$

Como $r_1 < 2$, también se cumple $T(n) = O(2^n)$.

Ejemplo: Fibonacci recursivo (costo espacial) I

Análisis de consumo de memoria

- ▶ Cada llamada recursiva consume una unidad de memoria en la pila de ejecución.
- ▶ La profundidad máxima de la recursión es n (camino $F(n) \rightarrow F(n-1) \rightarrow \dots \rightarrow F(0)$).
- ▶ Variables locales por llamada: constante (parámetro n y temporales).

El consumo espacial está determinado por la profundidad de la pila:

$$M(n) = \Theta(n).$$

Ejemplo: Fibonacci iterativo (enunciado) I

Problema. Calcular el n -ésimo término de la sucesión de Fibonacci utilizando un algoritmo iterativo que evite las llamadas recursivas redundantes. Determine la complejidad temporal y espacial de la implementación iterativa óptima.

Ejemplo: Fibonacci iterativo (algoritmo) I

Algoritmo 4 FibIterativo(n)

```
1: function FIBITERATIVO( $n$ )
2:   if  $n \leq 1$  then
3:     return  $n$ 
4:   end if
5:    $a \leftarrow 0, b \leftarrow 1$ 
6:   for  $i = 2$  to  $n$  do
7:      $c \leftarrow a + b$ 
8:      $a \leftarrow b$ 
9:      $b \leftarrow c$ 
10:  end for
11:  return  $b$ 
12: end function
```

Ejemplo: Fibonacci iterativo (análisis de operaciones) I

Conteo detallado de operaciones elementales:

Línea	Operación	Tipo	Costo	Frecuencia
2	$n \leq 1$	Comparación	1	1
3	return n	Retorno	1	≤ 1
5	$a \leftarrow 0, b \leftarrow 1$	Asignaciones	2	1
6	$i \leftarrow 2$	Asignación	1	1
6	$i \leq n$	Comparación	1	$n - 1$
7	$a + b$	Suma	1	$n - 1$
7	$c \leftarrow \dots$	Asignación	1	$n - 1$
8	$a \leftarrow b$	Asignación	1	$n - 1$
9	$b \leftarrow c$	Asignación	1	$n - 1$
6	$i \leftarrow i + 1$	Asignación+suma	2	$n - 1$
11	return b	Retorno	1	1

Ejemplo: Fibonacci iterativo (costo temporal) I

El costo temporal total es:

$T(n) = 1 \cdot 1$	(comparación inicial)
$+ 1 \cdot 1$	(retorno caso base, si aplica)
$+ 2 \cdot 1$	(inicialización a, b)
$+ 1 \cdot 1$	(inicialización i)
$+ 1 \cdot (n - 1)$	(comparaciones bucle)
$+ 1 \cdot (n - 1)$	(sumas)
$+ 1 \cdot (n - 1)$	(asignación c)
$+ 1 \cdot (n - 1)$	(asignación a)
$+ 1 \cdot (n - 1)$	(asignación b)
$+ 2 \cdot (n - 1)$	(incremento i)
$+ 1 \cdot 1$	(retorno final)

Ejemplo: Fibonacci iterativo (costo temporal) II

$$\begin{aligned}T(n) &= 1 + 1 + 2 + 1 + (n - 1) + (n - 1) + \\&\quad + (n - 1) + (n - 1) + (n - 1) + 2(n - 1) + 1 \\&= 7n - 1.\end{aligned}$$

Para $n \geq 1$:

$$7n - 1 \leq 7n.$$

Por lo tanto, con $c = 7$ y $n_0 = 1$:

$$T(n) = O(n).$$

Ejemplo: Fibonacci iterativo (costo espacial) I

Análisis de consumo de memoria

- ▶ Variable n : entrada (1 unidad).
- ▶ Variables a , b , c : acumuladores (3 unidades).
- ▶ Variable i : contador de bucle (1 unidad).
- ▶ Código del algoritmo: memoria constante.

El consumo total es independiente de n :

$$M(n) = 5 + \text{constante} = O(1).$$

El algoritmo presenta complejidad espacial constante.

Ejemplo: comparación de implementaciones I

Implementación	Tiempo	Espacio
Recursiva directa	$\Theta(\phi^n)$, $\phi = \frac{1+\sqrt{5}}{2}$	$\Theta(n)$
Iterativa	$\Theta(n)$	$O(1)$

Conclusión algorítmica

La implementación recursiva directa presenta un crecimiento exponencial del tiempo de ejecución debido a la recomputación masiva de subproblemas idénticos. La versión iterativa elimina esta redundancia mediante un enfoque de cálculo progresivo con estado constante, logrando una mejora asintótica drástica: de tiempo exponencial a lineal y de espacio lineal a constante. Este ejemplo ilustra cómo la estructura algorítmica subyacente determina fundamentalmente la eficiencia asintótica, más allá de consideraciones de implementación específica.

Comparación conceptual: búsqueda lineal vs. binaria I

Comparar las complejidades temporales y espaciales de los algoritmos de búsqueda lineal y búsqueda binaria (iterativa y recursiva) sobre un arreglo ordenado de n elementos.

Analice el impacto de la estructura algorítmica (iteración vs. recursión) en el consumo de recursos computacionales.

Comparación conceptual: análisis I

Complejidad temporal

- ▶ Búsqueda lineal: $O(n)$ en peor caso (recorrido secuencial completo).
- ▶ Búsqueda binaria (iterativa y recursiva): $O(\log n)$ en peor caso (división sucesiva del espacio de búsqueda).

Complejidad espacial

- ▶ Búsqueda lineal (iterativa): $O(1)$ (variables auxiliares constantes).
- ▶ Búsqueda binaria (iterativa): $O(1)$ (índices *izq*, *der*, *m*).
- ▶ Búsqueda binaria (recursiva): $O(\log n)$ (profundidad de la pila de recursión).

Comparación conceptual: análisis II

El análisis demuestra que la recursión introduce un coste espacial adicional proporcional a la profundidad de llamadas, mientras que las versiones iterativas mantienen consumo constante independiente de n .

Búsqueda lineal: enunciado I

Localizar la posición de un elemento x en un arreglo $A[0 \dots n - 1]$ mediante exploración secuencial.

Determine las complejidades temporal (mejor y peor caso) y espacial del algoritmo iterativo.

Búsqueda lineal: algoritmo I

Algoritmo 5 Lineal(A, n, x)

```
1: function LINEAL( $A, n, x$ )
2:   for  $i = 0$  to  $n - 1$  do
3:     if  $A[i] = x$  then
4:       return  $i$ 
5:     end if
6:   end for
7:   return  $-1$ 
8: end function
```

Búsqueda lineal: análisis de operaciones I

Línea	Operación	Tipo	Costo	Frecuencia
2	$i \leftarrow 0$	Asignación	1	1
2	$i \leq n - 1$	Comparación	1	$n + 1$
3	$A[i] = x$	Comparación	1	n
4	return i	Retorno	1	≤ 1
2	$i \leftarrow i + 1$	Asignación+suma	2	n
7	return -1	Retorno	1	1

Búsqueda lineal: costo temporal (mejor caso) I

El mejor caso ocurre cuando x se encuentra en la primera posición ($i = 0$):

$$T_{\text{mejor}}(n) = 1 + 1 + 1 + 1 = 4.$$

Por lo tanto,

$$T_{\text{mejor}}(n) = \Theta(1).$$

Búsqueda lineal: costo temporal (peor caso) I

El peor caso ocurre cuando x no pertenece al arreglo o está en la última posición:

$$\begin{aligned}T_{\text{peor}}(n) &= 1 \cdot 1 + 1 \cdot (n + 1) + 1 \cdot n + 1 \cdot 0 + 2 \cdot n + 1 \cdot 1 \\&= 1 + n + 1 + n + 2n + 1 \\&= 4n + 3.\end{aligned}$$

Para $n \geq 1$:

$$4n + 3 \leq 4n + 3n = 7n.$$

Por lo tanto, con $c = 7$ y $n_0 = 1$:

$$T_{\text{peor}}(n) = O(n).$$

Búsqueda lineal: costo espacial I

Análisis de memoria

- ▶ Arreglo A : n posiciones (entrada, no contabilizada en el coste adicional).
- ▶ Variable x : 1 unidad.
- ▶ Variable i : 1 unidad.
- ▶ Código del algoritmo: memoria constante.

El consumo adicional es independiente de n :

$$M(n) = 2 + \text{constante} = O(1).$$

Búsqueda binaria iterativa: enunciado I

Localizar la posición de un elemento x en un arreglo ordenado $A[0 \dots n - 1]$ mediante división sucesiva del intervalo de búsqueda.

Determine las complejidades temporal y espacial de la implementación iterativa.

Búsqueda binaria iterativa: algoritmo I

Algoritmo 6 Binarialter(A, n, x)

```
1: function BINARIALTER( $A, n, x$ )
2:    $izq \leftarrow 0, der \leftarrow n - 1$ 
3:   while  $izq \leq der$  do
4:      $m \leftarrow \lfloor (izq + der)/2 \rfloor$ 
5:     if  $A[m] = x$  then
6:       return  $m$ 
7:     else if  $A[m] < x$  then
8:        $izq \leftarrow m + 1$ 
9:     else
10:       $der \leftarrow m - 1$ 
11:    end if
12:  end while
13:  return  $-1$ 
14: end function
```

Búsqueda binaria iterativa: análisis de operaciones I

Línea	Operación	Tipo	Costo	Frecuencia
2	$izq \leftarrow 0, der \leftarrow n - 1$	Asignaciones	2	1
3	$izq \leq der$	Comparación	1	$\lfloor \log_2 n \rfloor + 1$
4	$m \leftarrow \lfloor (izq + der)/2 \rfloor$	Asign.+suma+div.+piso	4	$\lfloor \log_2 n \rfloor$
5	$A[m] = x$	Comparación	1	$\lfloor \log_2 n \rfloor$
6	return m	Retorno	1	≤ 1
7	$A[m] < x$	Comparación	1	$\leq \lfloor \log_2 n \rfloor$
8	$izq \leftarrow m + 1$	Asign.+suma	2	$\leq \lfloor \log_2 n \rfloor$
10	$der \leftarrow m - 1$	Asign.+resta	2	$\leq \lfloor \log_2 n \rfloor$
13	return -1	Retorno	1	1

Búsqueda binaria iterativa: costo temporal I

En el peor caso (elemento no encontrado), todas las operaciones se ejecutan el número máximo de veces:

$$\begin{aligned}T(n) &= 2 \cdot 1 + 1 \cdot (\lfloor \log_2 n \rfloor + 1) + 4 \cdot \lfloor \log_2 n \rfloor + 1 \cdot \lfloor \log_2 n \rfloor \\&\quad + 1 \cdot 0 + 1 \cdot \lfloor \log_2 n \rfloor + 2 \cdot \lfloor \log_2 n \rfloor + 2 \cdot \lfloor \log_2 n \rfloor + 1 \cdot 1 \\&= 2 + \lfloor \log_2 n \rfloor + 1 + 4\lfloor \log_2 n \rfloor + \lfloor \log_2 n \rfloor + \lfloor \log_2 n \rfloor \\&\quad + 2\lfloor \log_2 n \rfloor + 2\lfloor \log_2 n \rfloor + 1 \\&= 11\lfloor \log_2 n \rfloor + 4.\end{aligned}$$

Para $n \geq 2$:

$$11\lfloor \log_2 n \rfloor + 4 \leq 11\log_2 n + 4 \leq 11\log_2 n + 4\log_2 n = 15\log_2 n.$$

Por lo tanto:

$$T(n) = O(\log n).$$

Búsqueda binaria iterativa: costo espacial I

Análisis de memoria

- ▶ Arreglo A : n posiciones (entrada).
- ▶ Variables x , izq , der , m : 4 unidades.
- ▶ Código del algoritmo: memoria constante.

El consumo adicional es constante:

$$M(n) = 4 + \text{constante} = O(1).$$

Búsqueda binaria recursiva: enunciado I

Localizar la posición de un elemento x en un arreglo ordenado $A[0 \dots n - 1]$ mediante recursión que divide el intervalo de búsqueda.

Determine las complejidades temporal y espacial de la implementación recursiva.

Búsqueda binaria recursiva: algoritmo I

Algoritmo 7 BinariaRec(A, x, izq, der)

```
1: function BINARIAREC( $A, x, izq, der$ )
2:   if  $izq > der$  then
3:     return  $-1$ 
4:   end if
5:    $m \leftarrow \lfloor (izq + der)/2 \rfloor$ 
6:   if  $A[m] = x$  then
7:     return  $m$ 
8:   else if  $A[m] < x$  then
9:     return BINARIAREC( $A, x, m + 1, der$ )
10:  else
11:    return BINARIAREC( $A, x, izq, m - 1$ )
12:  end if
13: end function
```

Búsqueda binaria recursiva: análisis de operaciones I

Línea	Operación	Tipo	Costo	Frecuencia
2	$izq > der$	Comparación	1	1 por llamada
3	return -1	Retorno	1	1 (última llamada)
5	$m \leftarrow \lfloor (izq + der)/2 \rfloor$	Asign.+suma+div.+piso	4	1 por llamada
6	$A[m] = x$	Comparación	1	1 por llamada
7	return m	Retorno	1	≤ 1
8	$A[m] < x$	Comparación	1	≤ 1 por llamada
9,11	Llamada recursiva	Gestión de pila	$T(n/2)$	1 por llamada

Búsqueda binaria recursiva: costo temporal I

La función de costo satisface la recurrencia:

$$T(n) = \begin{cases} 2 & \text{si } n = 1, \\ T(n/2) + 7 & \text{si } n > 1. \end{cases}$$

Desarrollando la recurrencia:

$$\begin{aligned} T(n) &= T(n/2) + 7 \\ &= T(n/4) + 7 + 7 \\ &= \dots \\ &= T(1) + 7 \log_2 n \\ &= 2 + 7 \log_2 n. \end{aligned}$$

Búsqueda binaria recursiva: costo temporal II

Para $n \geq 2$:

$$2 + 7 \log_2 n \leq 2 \log_2 n + 7 \log_2 n = 9 \log_2 n.$$

Por lo tanto:

$$T(n) = O(\log n).$$

Búsqueda binaria recursiva: costo espacial I

Análisis de memoria

- ▶ Cada llamada recursiva consume una trama en la pila de ejecución con tamaño constante (parámetros y variables locales).
- ▶ La profundidad máxima de recursión es $\lfloor \log_2 n \rfloor + 1$ (número de divisiones hasta intervalo vacío).

El consumo espacial total es proporcional a la profundidad de la pila:

$$M(n) = \Theta(\log n).$$

Tabla comparativa de algoritmos de búsqueda I

Algoritmo	Tiempo	Espacio	Características
Búsqueda lineal	$O(n)$	$O(1)$	No requiere arreglo ordenado; óptima para datos pequeños o no ordenados.
Búsqueda binaria (iterativa)	$O(\log n)$	$O(1)$	Requiere arreglo ordenado; mínima sobrecarga espacial.
Búsqueda binaria (recursiva)	$O(\log n)$	$O(\log n)$	Requiere arreglo ordenado; elegante pero con coste espacial por recursión.

Tabla comparativa de algoritmos de búsqueda II

Conclusión

La búsqueda binaria ofrece ventaja asintótica temporal sobre la búsqueda lineal para arreglos ordenados de tamaño considerable. La versión iterativa es preferible en entornos con restricciones de memoria, mientras que la recursiva proporciona mayor claridad conceptual a cambio de un coste espacial logarítmico.

Análisis de crecimiento de funciones

Introducción al análisis de crecimiento asintótico I

Fundamentación teórica

El análisis del crecimiento relativo de funciones constituye la base matemática para la clasificación rigurosa de algoritmos según su eficiencia computacional. Este análisis permite:

- ▶ Establecer jerarquías formales entre clases de complejidad.
- ▶ Predecir el comportamiento algorítmico ante entradas de tamaño creciente.
- ▶ Identificar umbrales prácticos donde una estrategia computacional supera a otra.

Introducción al análisis de crecimiento asintótico II

Funciones prototípicas fundamentales

Las funciones que aparecen recurrentemente en el análisis de algoritmos forman una secuencia jerárquica:

$$1, \log \log n, \log n, (\log n)^k, n^a, n^a \log^b n, a^n, n!, n^n$$

donde $k > 0$, $0 < a < b$ y $a > 1$ son constantes reales.

El objetivo de esta sección es demostrar formalmente las relaciones de dominancia asintótica que establecen el orden estricto de crecimiento entre estas funciones fundamentales.

Jerarquía asintótica fundamental

Jerarquía asintótica fundamental I

Relación de dominancia estricta

Existe un orden estricto de crecimiento entre las funciones prototípicas fundamentales:

$$1 < \log \log n < \log n < (\log n)^k < n^a < n^b < a^n < b^n < n! < n^n$$

para constantes $k > 0$, $0 < a < b$ y $a, b > 1$.

Jerarquía asintótica fundamental II

Caracterización mediante límites

Para cada par consecutivo $f(n), g(n)$ en la secuencia anterior se verifica:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \quad \Longleftrightarrow \quad f(n) = o(g(n)).$$

Esta relación implica $f(n) = O(g(n))$ pero $g(n) \notin O(f(n))$, estableciendo una dominancia asintótica estricta.

Teorema 1: dominancia exponencial sobre polinómica

Teorema 1: dominancia exponencial sobre polinómica I

Enunciado formal

Sean $a > 1$ y $b > 0$ constantes reales. Entonces existe $x_0 > 0$ tal que

$$a^x \geq x^b, \quad \forall x \geq x_0.$$

En términos asintóticos:

$$x^b = o(a^x).$$

Interpretación algorítmica

Los algoritmos con complejidad polinómica $O(n^b)$ escalan significativamente mejor que aquellos con complejidad exponencial $O(a^n)$, incluso para exponentes polinómicos grandes y bases exponenciales cercanas a 1. Esta diferencia se vuelve abismal para entradas de tamaño moderado o grande.

Teorema 1: demostración I

Defínase la función auxiliar

$$f(x) = \frac{b \log x}{x \log a}, \quad \forall x > 0.$$

Su derivada es

$$f'(x) = \left(\frac{b}{\log a} \right) \frac{1 - \log x}{x^2}.$$

Por lo tanto,

$$f'(x) = \begin{cases} > 0 & \text{si } 0 < x < e, \\ \leq 0 & \text{si } x \geq e. \end{cases}$$

La función f alcanza su máximo global en $x = e$ y es estrictamente decreciente en $[e, +\infty)$.

Teorema 1: demostración II

Además,

$$\lim_{x \rightarrow +\infty} f(x) = \left(\frac{b}{\log a} \right) \lim_{x \rightarrow +\infty} \frac{\log x}{x} \stackrel{\text{L'Hôpital}}{=} 0.$$

Como f es continua, positiva y decreciente en $[e, +\infty)$ con límite nulo, existe $x_0 \geq e$ tal que $f(x) \leq 1$ para todo $x \geq x_0$.

Por lo tanto,

$$\frac{b \log x}{x \log a} \leq 1 \implies b \log x \leq x \log a \implies a^x \geq x^b, \quad \forall x \geq x_0.$$



Ilustración: crecimiento exponencial vs. polinómico I

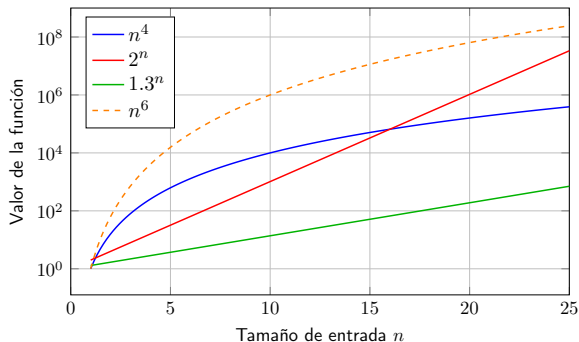


Ilustración: crecimiento exponencial vs. polinómico II

Observación visual

Aunque los polinomios de alto grado (n^6) superan inicialmente a las exponenciales de base cercana a 1 (1.3^n), toda función exponencial con base $a > 1$ eventualmente domina a cualquier polinomio n^b . El punto de cruce entre n^4 y 2^n ocurre en $n \approx 16$, ilustrando que la dominancia asintótica se manifiesta para valores prácticos de n .

Teorema 2: dominancia polinómica sobre logarítmica

Teorema 2: dominancia polinómica sobre logarítmica I

Enunciado formal

Sea $a > 0$ una constante real. Entonces existe $x_0 > 0$ tal que

$$x^a \geq \log x, \quad \forall x \geq x_0.$$

En términos asintóticos:

$$\log x = o(x^a).$$

Interpretación algorítmica

Los algoritmos con complejidad logarítmica $O(\log n)$ presentan una escalabilidad superior a los polinómicos $O(n^a)$ para cualquier $a > 0$, aunque la ventaja relativa es más pronunciada para exponentes a grandes. Esta propiedad justifica el uso intensivo de estructuras de datos basadas en árboles balanceados y búsquedas binarias en aplicaciones de alto rendimiento.

Teorema 2: demostración I

Defínase la función auxiliar

$$f(x) = \frac{\log x}{x^a}, \quad \forall x > 0.$$

Su derivada es

$$f'(x) = \frac{1 - a \log x}{x^{a+1}}.$$

Por lo tanto,

$$f'(x) = \begin{cases} > 0 & \text{si } 0 < x < e^{1/a}, \\ \leq 0 & \text{si } x \geq e^{1/a}. \end{cases}$$

La función f alcanza su máximo global en $x = e^{1/a}$ y es estrictamente decreciente en $[e^{1/a}, +\infty)$.

Teorema 2: demostración II

Además,

$$\lim_{x \rightarrow +\infty} f(x) = \lim_{x \rightarrow +\infty} \frac{\log x}{x^a} \stackrel{\text{L'Hôpital}}{=} \lim_{x \rightarrow +\infty} \frac{1}{ax^a} = 0.$$

Como f es continua, positiva y decreciente en $[e^{1/a}, +\infty)$ con límite nulo, existe $x_0 \geq e^{1/a}$ tal que $f(x) \leq 1$ para todo $x \geq x_0$.

Por lo tanto,

$$\frac{\log x}{x^a} \leq 1 \implies x^a \geq \log x, \quad \forall x \geq x_0.$$



Ilustración: crecimiento polinómico vs. logarítmico I

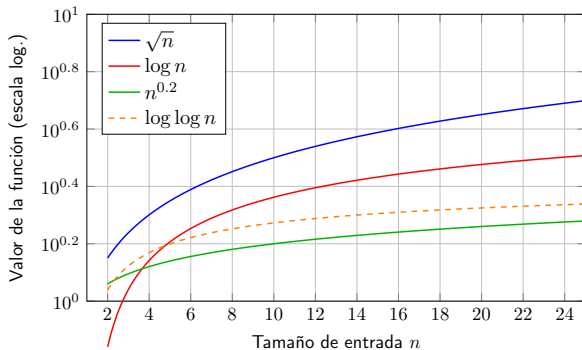


Ilustración: crecimiento polinómico vs. logarítmico II

Observación visual

Aunque $\log n$ crece inicialmente más rápido que $n^{0.2}$ para valores muy pequeños de n , cualquier potencia positiva n^a con $a > 0$ eventualmente supera a $\log n$. La función $\log \log n$ crece extremadamente lentamente, permaneciendo por debajo de 5 incluso para $n = 10^{50}$, lo que la hace prácticamente constante en aplicaciones reales.

Casos clásicos adicionales I

Dominancia factorial sobre exponencial

Para cualquier constante $a > 1$ existe n_0 tal que

$$n! \geq a^n, \quad \forall n \geq n_0.$$

Esto se demuestra observando que para $n > \lceil a \rceil \geq a$:

$$\begin{aligned} n! &= \underbrace{1 \cdot 2 \cdots \lceil a \rceil}_{\text{constante}} \cdot \underbrace{(\lceil a \rceil + 1) \cdots (\lceil a \rceil + (n - \lceil a \rceil))}_{> a \text{ en cada factor}} \\ &> \lceil a \rceil! \cdot a^{n - \lceil a \rceil} = \left(\frac{\lceil a \rceil!}{a^{\lceil a \rceil}} \right) a^n. \end{aligned}$$

Por lo tanto $a^n = o(n!)$.

Casos clásicos adicionales II

Dominancia doble logarítmica

Para cualquier constante $a > 0$ existe n_0 tal que

$$\log n \geq (\log \log n)^a, \quad \forall n \geq n_0.$$

Esto se verifica mediante el cambio de variable $m = \log n$, reduciendo el problema a demostrar $m \geq (\log m)^a$, que es una aplicación directa del Teorema 2.

Ilustración: crecimiento factorial vs. exponencial I

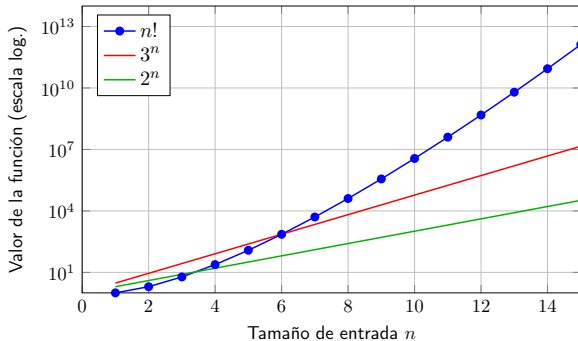


Ilustración: crecimiento factorial vs. exponencial II

Observación visual

Aunque 3^n supera inicialmente a $n!$ para $n \leq 6$, el factorial domina rápidamente a partir de $n = 7$. Esta transición temprana ilustra por qué los algoritmos con complejidad factorial son prácticamente intratables incluso para instancias modestas ($n \geq 20$), mientras que los exponenciales pueden ser viables para n hasta 40-50 en hardware moderno.

Teorema 3: jerarquía asintótica completa

Teorema 3: jerarquía asintótica completa I

Enunciado formal

Sean $k > 0$, $0 < a < b$ y $c, d > 1$ constantes reales. Entonces existe $n_0 > 0$ tal que para todo $n \geq n_0$:

$$1 < \log \log n < \log n < (\log n)^k < n^a < n^b < c^n < d^n < n! < n^n.$$

Consecuencia asintótica

Cada función en la secuencia es o de la siguiente:

$$1 = o(\log \log n), \log \log n = o(\log n), \log n = o((\log n)^k), \dots, n! = o(n^n).$$

Este orden estricto fundamenta la clasificación teórica de algoritmos y explica las diferencias prácticas en escalabilidad computacional.

Teorema 3: demostración y conclusión I

La demostración se obtiene combinando los Teoremas 1 y 2 con los casos clásicos adicionales:

- ▶ $1 = o(\log \log n)$: sigue de $\log \log n \rightarrow \infty$ cuando $n \rightarrow \infty$.
- ▶ $\log \log n = o(\log n)$: aplicación del Teorema 2 con $x = \log n$ y $a = 1$.
- ▶ $\log n = o((\log n)^k)$ para $k > 1$: evidente por $(\log n)^{k-1} \rightarrow \infty$.
- ▶ $(\log n)^k = o(n^a)$: aplicación del Teorema 2 con $x = n$ y exponente $a/k > 0$.
- ▶ $n^a = o(n^b)$ para $0 < a < b$: evidente por $n^{b-a} \rightarrow \infty$.
- ▶ $n^b = o(c^n)$: Teorema 1 con $x = n$, base $c > 1$ y exponente b .
- ▶ $c^n = o(d^n)$ para $1 < c < d$: evidente por $(d/c)^n \rightarrow \infty$.
- ▶ $d^n = o(n!)$: caso clásico demostrado anteriormente.
- ▶ $n! = o(n^n)$: evidente por $n! = 1 \cdot 2 \cdots n < n \cdot n \cdots n = n^n$.

Teorema 3: demostración y conclusión II

Por transitividad de la relación $o(\cdot)$, se obtiene la jerarquía completa.



Implicación práctica fundamental

Esta jerarquía establece que la diferencia entre clases adyacentes en la secuencia representa un salto cualitativo en escalabilidad. Un algoritmo con complejidad $O(n \log n)$ escalará órdenes de magnitud mejor que uno con $O(n^2)$ para entradas grandes, y la diferencia entre $O(2^n)$ y $O(n!)$ es aún más dramática. El análisis asintótico riguroso es esencial para la selección de algoritmos en aplicaciones reales.

Análisis de complejidad en algoritmos clásicos

Búsqueda lineal: enunciado del problema I

Problema

Localizar la posición de un elemento x en un arreglo $A[0 \dots n - 1]$ de n elementos mediante exploración secuencial.

Características:

- ▶ No requiere que el arreglo esté ordenado.
- ▶ Examina los elementos en orden secuencial hasta encontrar el objetivo o agotar el arreglo.

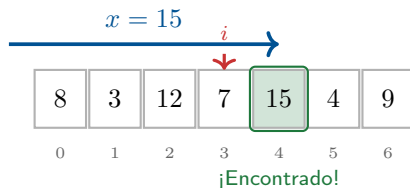
Determine las complejidades temporales en los casos mejor, promedio y peor, así como la complejidad espacial del algoritmo iterativo.

Búsqueda lineal: algoritmo

Algoritmo 8 Lineal(A, n, x)

```
1: function LINEAL( $A, n, x$ )
2:   for  $i = 0$  to  $n - 1$  do
3:     if  $A[i] = x$  then
4:       return  $i$ 
5:     end if
6:   end for
7:   return  $-1$ 
8: end function
```

Búsqueda lineal: elemento encontrado



Recorrido secuencial hasta localizar x en $A[4]$

Búsqueda lineal: análisis de complejidad temporal I

- ▶ **Mejor caso:** El elemento x se encuentra en la primera posición ($i = 0$). El algoritmo realiza una única comparación:

$$T_{\text{mejor}}(n) = \Theta(1).$$

- ▶ **Peor caso:** El elemento x no pertenece al arreglo o se encuentra en la última posición ($i = n - 1$). El algoritmo examina todos los n elementos:

$$T_{\text{peor}}(n) = \Theta(n).$$

- ▶ **Caso promedio:** Asumiendo distribución uniforme sobre las $n + 1$ posibilidades (las n posiciones más la ausencia), el número esperado de comparaciones es:

$$T_{\text{prom}}(n) = \frac{1}{n+1} \sum_{k=1}^n k + \frac{n}{n+1} = \frac{n+2}{2} = \Theta(n).$$

Búsqueda lineal: análisis de complejidad temporal II

Complejidad asintótica

La complejidad temporal en notación O se determina por el peor caso:

$$T(n) = O(n).$$

La complejidad espacial es constante, ya que el algoritmo utiliza únicamente variables auxiliares independientes de n :

$$M(n) = O(1).$$

Búsqueda binaria: enunciado del problema I

Problema

Localizar la posición de un elemento x en un arreglo ordenado $A[0 \dots n - 1]$ de n elementos mediante división sucesiva del intervalo de búsqueda.

Características:

- ▶ Requiere que el arreglo esté ordenado en forma no decreciente.
- ▶ En cada iteración compara x con el elemento central del intervalo actual y descarta la mitad del espacio de búsqueda.

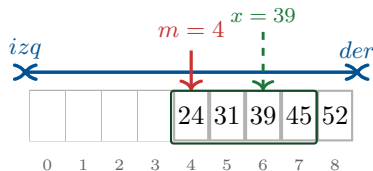
Determine la complejidad temporal en el peor caso y la complejidad espacial de las versiones iterativa y recursiva.

Búsqueda binaria: algoritmo iterativo

Algoritmo 9 BinariaIter(A, n, x)

```
1: function BINARIAITER( $A, n, x$ )
2:    $izq \leftarrow 0, der \leftarrow n - 1$ 
3:   while  $izq \leq der$  do
4:      $m \leftarrow \lfloor (izq + der) / 2 \rfloor$ 
5:     if  $A[m] = x$  then
6:       return  $m$ 
7:     else if  $A[m] < x$  then
8:        $izq \leftarrow m + 1$ 
9:     else
10:       $der \leftarrow m - 1$ 
11:    end if
12:  end while
13:  return  $-1$ 
14: end function
```

Búsqueda binaria: ejemplo de ejecución



Descarte de mitad izquierda tras comparar $A[4] = 24 < x$

Búsqueda binaria: análisis de complejidad temporal I

Ecuación de recurrencia

En el peor caso, el algoritmo reduce el tamaño del problema a la mitad en cada iteración y realiza un número constante c de operaciones elementales (comparaciones, cálculo del punto medio, actualización de índices):

$$T(n) = T\left(\frac{n}{2}\right) + c, \quad T(1) = d,$$

donde $c, d > 0$ son constantes.

Búsqueda binaria: análisis de complejidad temporal II

Resolución de la recurrencia

Desarrollando la recurrencia mediante expansión iterativa:

$$\begin{aligned}T(n) &= T\left(\frac{n}{2}\right) + c = T\left(\frac{n}{4}\right) + 2c \\&= T\left(\frac{n}{8}\right) + 3c = \dots = T\left(\frac{n}{2^k}\right) + kc.\end{aligned}$$

El proceso termina cuando $\frac{n}{2^k} = 1$, es decir, $k = \log_2 n$. Sustituyendo:

$$T(n) = T(1) + c \log_2 n = d + c \log_2 n = \Theta(\log n).$$

Por lo tanto,

$$T(n) = O(\log n).$$

Búsqueda binaria: análisis de complejidad espacial I

Versión iterativa

El algoritmo utiliza únicamente tres variables enteras (*izq*, *der*, *m*) además del arreglo de entrada. El consumo adicional de memoria es independiente de n :

$$M_{\text{iter}}(n) = O(1).$$

Versión recursiva

Cada llamada recursiva consume una trama en la pila de ejecución con tamaño constante. La profundidad máxima de recursión es $\lfloor \log_2 n \rfloor + 1$, correspondiente al número de divisiones hasta alcanzar un intervalo unitario:

$$M_{\text{rec}}(n) = \Theta(\log n).$$

Búsqueda binaria: análisis de complejidad espacial II

Conclusión comparativa

Ambas versiones presentan idéntica complejidad temporal $O(\log n)$, pero la versión iterativa es preferible en entornos con restricciones de memoria debido a su consumo espacial constante frente al logarítmico de la versión recursiva.

Ordenamiento burbuja: enunciado del problema I

Problema

Ordenar un arreglo $A[0 \dots n - 1]$ de n elementos en orden no decreciente mediante comparaciones e intercambios sucesivos de elementos adyacentes.

Características:

- ▶ Algoritmo de ordenamiento por comparación estable.
- ▶ Requiere dos bucles anidados que recorren el arreglo comparando elementos adyacentes.
- ▶ Tras cada pasada completa del bucle externo, el elemento máximo restante "burbujea" hasta su posición final.

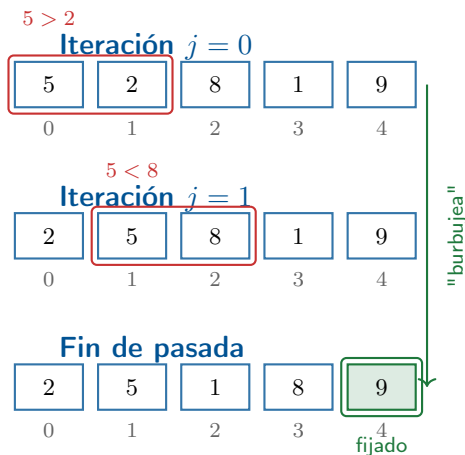
Determine la complejidad temporal en el peor caso y la complejidad espacial del algoritmo.

Ordenamiento burbuja: algoritmo I

Algoritmo 10 Burbuja(A, n)

```
1: function BURBUJA( $A, n$ )
2:   for  $i = 0$  to  $n - 2$  do
3:     for  $j = 0$  to  $n - i - 2$  do
4:       if  $A[j] > A[j + 1]$  then
5:         Intercambiar  $A[j]$  y  $A[j + 1]$ 
6:       end if
7:     end for
8:   end for
9:   return  $A$ 
10: end function
```

Ordenamiento burbuja: algoritmo II



Ordenamiento burbuja: análisis de operaciones I

Línea	Operación	Tipo	Costo	Frecuencia
2	$i \leftarrow 0$	Asignación	1	1
2	$i \leq n - 2$	Comparación	1	n
3	$j \leftarrow 0$	Asignación	1	$n - 1$
3	$j \leq n - i - 2$	Comparación	1	$\sum_{i=0}^{n-2} (n - i - 1) + 1$
4	$A[j] > A[j + 1]$	Comparación	1	$\sum_{i=0}^{n-2} (n - i - 1)$
5	Intercambio (3 asign.)	Asignaciones	3	$\leq \sum_{i=0}^{n-2} (n - i - 1)$
3	$j \leftarrow j + 1$	Asign.+suma	2	$\sum_{i=0}^{n-2} (n - i - 1)$
2	$i \leftarrow i + 1$	Asign.+suma	2	$n - 1$
9	return A	Retorno	1	1

Ordenamiento burbuja: análisis de operaciones II

Definiendo la sumatoria auxiliar:

$$S = \sum_{i=0}^{n-2} (n - i - 1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}.$$

Ordenamiento burbuja: costo temporal I

El costo temporal total se obtiene sumando las contribuciones:

$$\begin{aligned}T(n) &= 1 \cdot 1 + 1 \cdot n + 1 \cdot (n - 1) + 1 \cdot (S + 1) + 1 \cdot S + \\&\quad + 3 \cdot S + 2 \cdot S + 2 \cdot (n - 1) + 1 \cdot 1 \\&= 1 + n + n - 1 + S + 1 + S + 3S + 2S + 2n - 2 + 1 \\&= 7S + 4n \\&= 7 \cdot \frac{n(n - 1)}{2} + 4n \\&= \frac{7n^2 - 7n + 8n}{2} \\&= \frac{7n^2 + n}{2}.\end{aligned}$$

Para $n \geq 1$:

$$\frac{7n^2 + n}{2} \leq \frac{7n^2 + n^2}{2} = 4n^2.$$

Ordenamiento burbuja: costo temporal II

Por lo tanto, con $c = 4$ y $n_0 = 1$:

$$T(n) = O(n^2).$$

Complejidad espacial

El algoritmo opera *in situ* (modifica el arreglo original sin estructuras auxiliares de tamaño proporcional a n). El consumo adicional se limita a las variables i , j y una temporal para el intercambio:

$$M(n) = O(1).$$

Ordenamiento por mezcla: enunciado del problema I

Problema

Ordenar un arreglo $A[0 \dots n - 1]$ de n elementos en orden no decreciente mediante la técnica de divide y vencerás.

Estrategia algorítmica:

- ▶ **Dividir:** Partir el arreglo en dos subarreglos de tamaño aproximadamente igual.
- ▶ **Vencer:** Ordenar recursivamente cada subarreglo.
- ▶ **Combinar:** Mezclar los dos subarreglos ordenados en un único arreglo ordenado mediante un procedimiento lineal.

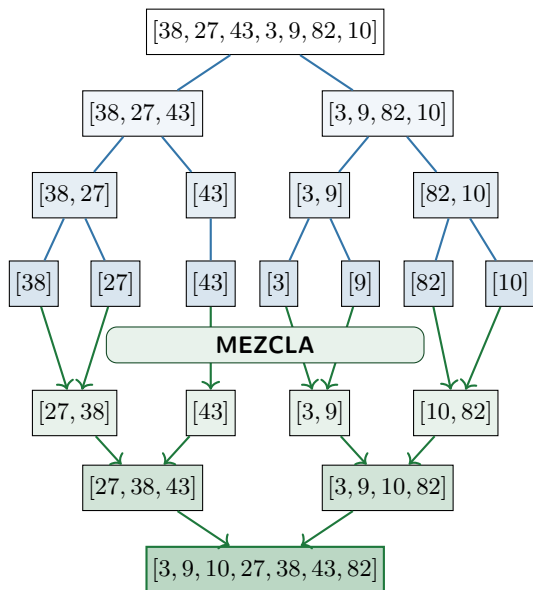
Determine la complejidad temporal y espacial del algoritmo en el peor caso.

Ordenamiento por mezcla: algoritmo I

Algoritmo 11 MergeSort(A, n)

```
1: function MERGESORT( $A, n$ )
2:   if  $n \leq 1$  then
3:     return  $A$ 
4:   end if
5:    $m \leftarrow \lfloor n/2 \rfloor$ 
6:    $L \leftarrow A[0 \dots m - 1]$ 
7:    $R \leftarrow A[m \dots n - 1]$ 
8:    $L \leftarrow \text{MERGESORT}(L, m)$ 
9:    $R \leftarrow \text{MERGESORT}(R, n - m)$ 
10:  return MERGE( $L, R$ )
11: end function
```

Ordenamiento por mezcla: algoritmo II



Ordenamiento por mezcla: análisis de complejidad I

Ecuación de recurrencia

El algoritmo divide el problema de tamaño n en dos subproblemas de tamaño $\lfloor n/2 \rfloor$ y $\lceil n/2 \rceil$, resuelve cada uno recursivamente y luego combina los resultados mediante un procedimiento lineal de costo $\Theta(n)$:

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + cn, \quad T(1) = d,$$

donde $c, d > 0$ son constantes.

Para simplificar el análisis, asumimos $n = 2^k$ y obtenemos:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn.$$

Ordenamiento por mezcla: análisis de complejidad II

Resolución mediante teorema maestro

La recurrencia tiene la forma $T(n) = aT(n/b) + f(n)$ con $a = 2$, $b = 2$ y $f(n) = cn$. Calculamos:

$$n^{\log_b a} = n^{\log_2 2} = n.$$

Como $f(n) = \Theta(n^{\log_b a})$, aplicamos el caso 2 del teorema maestro:

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(n \log n).$$

Por lo tanto,

$$T(n) = O(n \log n).$$

Ordenamiento rápido: enunciado del problema I

Problema

Ordenar un arreglo $A[0 \dots n - 1]$ de n elementos en orden no decreciente mediante la técnica de divide y vencerás con particionamiento.

Estrategia algorítmica:

- ▶ **Particionar:** Seleccionar un elemento pivote y reorganizar el arreglo de modo que todos los elementos menores queden a la izquierda y los mayores a la derecha del pivote.
- ▶ **Vencer:** Ordenar recursivamente los subarreglos izquierdo y derecho del pivote.
- ▶ **Combinar:** No se requiere paso explícito de combinación, pues el particionamiento coloca el pivote en su posición final definitiva.

Ordenamiento rápido: enunciado del problema II

Determine las complejidades temporales en los casos mejor, promedio y peor, así como la complejidad espacial.

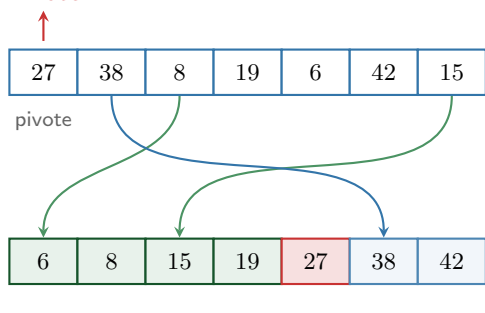
Ordenamiento rápido: algoritmo I

Algoritmo 12 QuickSort(A , izq , der)

```
1: function QUICKSORT( $A$ ,  $izq$ ,  $der$ )
2:   if  $izq < der$  then
3:      $p \leftarrow$  PARTICIONAR( $A$ ,  $izq$ ,  $der$ )
4:     QUICKSORT( $A$ ,  $izq$ ,  $p - 1$ )
5:     QUICKSORT( $A$ ,  $p + 1$ ,  $der$ )
6:   end if
7:   return  $A$ 
8: end function
```

Ordenamiento rápido: algoritmo II

Pivote



El pivote (27) alcanza su posición definitiva.

Los elementos se reorganizan a izquierda (menores) y derecha (mayores).

Ordenamiento rápido: análisis de complejidad I

Peor caso

Ocurre cuando el pivote seleccionado es sistemáticamente el mínimo o máximo elemento del subarreglo, produciendo particiones altamente desbalanceadas de tamaños 0 y $n - 1$. La recurrencia es:

$$T_{\text{peor}}(n) = T(n - 1) + cn, \quad T(1) = d.$$

Resolviendo por expansión:

$$T_{\text{peor}}(n) = cn + c(n - 1) + \cdots + c = c \frac{n(n + 1)}{2} = \Theta(n^2).$$

Ordenamiento rápido: análisis de complejidad II

Mejor caso

Ocurre cuando el pivote divide sistemáticamente el arreglo en dos mitades de tamaño aproximadamente igual. La recurrencia es:

$$T_{\text{mejor}}(n) = 2T\left(\frac{n}{2}\right) + cn = \Theta(n \log n),$$

por aplicación directa del teorema maestro (caso 2).

Ordenamiento rápido: análisis de complejidad III

Caso promedio

Bajo el supuesto de que todas las permutaciones de entrada son equiprobables, el pivote divide el arreglo en proporciones aleatorias. Se demuestra rigurosamente que:

$$T_{\text{prom}}(n) = \Theta(n \log n).$$

La complejidad espacial está determinada por la profundidad de la pila de recursión, que es $\Theta(\log n)$ en el caso promedio y $\Theta(n)$ en el peor caso.

Tabla comparativa de algoritmos de ordenamiento I

Algoritmo	Mejor caso	Promedio	Peor caso	Espacio
Burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Tabla comparativa de algoritmos de ordenamiento II

Observaciones comparativas

- ▶ El ordenamiento burbuja presenta complejidad cuadrática en la mayoría de los casos, lo que lo hace ineficiente para entradas de tamaño moderado o grande, a pesar de su simplicidad conceptual y bajo consumo espacial.
- ▶ El ordenamiento por mezcla garantiza complejidad $O(n \log n)$ en todos los casos a cambio de un consumo espacial lineal adicional para las operaciones de mezcla.
- ▶ El ordenamiento rápido ofrece excelente rendimiento promedio $O(n \log n)$ con bajo consumo espacial, pero su peor caso cuadrático puede ser problemático en aplicaciones críticas; variantes como introsort combinan quicksort con heapsort para garantizar $O(n \log n)$ en el peor caso.

Análisis de recurrencias

Ecuación característica

Ecuación característica: fundamentos teóricos I

Definición formal

La *ecuación característica* constituye un método sistemático para resolver recurrencias lineales con coeficientes constantes. Este procedimiento transforma un problema recursivo en uno algebraico mediante la construcción del polinomio característico asociado a la recurrencia homogénea.

Ecuación característica: fundamentos teóricos II

Esquema metodológico de resolución

El proceso de resolución se estructura en tres etapas secuenciales:

1. **Parte homogénea:** Resolver la recurrencia sin términos independientes ($= 0$), obteniendo el polinomio característico y sus raíces.
2. **Parte particular:** Proponer una forma tentativa para la solución particular basada en la naturaleza del término no homogéneo, ajustándola en caso de resonancia (cuando coincide con una raíz del polinomio característico).
3. **Solución general:** Combinar la solución homogénea con la particular y determinar las constantes mediante las condiciones iniciales de la recurrencia.

Caso 1: Recurrencias homogéneas

Recurrencias homogéneas: formulación general I

Forma canónica

Una recurrencia lineal homogénea de orden k con coeficientes constantes tiene la forma:

$$T(n) + a_1T(n-1) + \cdots + a_kT(n-k) = 0.$$

Construcción del polinomio característico

Se supone una solución de la forma $T(n) = r^n$ con $r \neq 0$. Sustituyendo en la recurrencia:

$$r^n + a_1r^{n-1} + \cdots + a_kr^{n-k} = 0.$$

Dividiendo por r^{n-k} se obtiene el polinomio característico:

$$r^k + a_1r^{k-1} + \cdots + a_k = 0.$$

Recurrencias homogéneas: formulación general II

Solución general según multiplicidades

Si las raíces (reales o complejas) son $r_1, \dots, r_\ell$ con multiplicidades $m_1, \dots, m_\ell$ tales que $\sum_{i=1}^{\ell} m_i = k$, entonces la solución general es:

$$T(n) = \sum_{i=1}^{\ell} q_i(n) r_i^n,$$

donde $q_i(n)$ es un polinomio de grado $m_i - 1$.

Recurrencias homogéneas: casos particulares I

Raíces simples

Quando todas las raíces son distintas ($m_i = 1$ para todo i), la solución general adopta la forma:

$$T(n) = \sum_{i=1}^k c_i r_i^n.$$

Raíces múltiples

Si una raíz r_i tiene multiplicidad $m_i > 1$, su contribución a la solución general es:

$$q_i(n) r_i^n = (c_{i1} + c_{i2}n + \cdots + c_{im_i}n^{m_i-1}) r_i^n.$$

Recurrencias homogéneas: casos particulares II

Raíces complejas conjugadas

Si $r = \alpha \pm \beta i$ es un par de raíces complejas conjugadas de multiplicidad m , su contribución puede expresarse en forma real mediante funciones trigonométricas:

$$\rho^n (A \cos(n\theta) + B \sin(n\theta)) ,$$

donde $\rho = \sqrt{\alpha^2 + \beta^2}$ y $\theta = \arctan(\beta/\alpha)$.

Ejemplo: sucesión de Fibonacci (enunciado) I

Problema. Analizar la recurrencia que define la sucesión de Fibonacci:

$$F_n = \begin{cases} 0 & \text{si } n = 0, \\ 1 & \text{si } n = 1, \\ F_{n-1} + F_{n-2} & \text{si } n \geq 2. \end{cases}$$

Determine una fórmula cerrada para F_n mediante el método de la ecuación característica y establezca su comportamiento asintótico.

Ejemplo: sucesión de Fibonacci (algoritmo recursivo)

Ejemplo: sucesión de Fibonacci (algoritmo recursivo) I

Algoritmo 13 FibRecursivo(n)

```
1: function FIBRECURSIVO( $n$ )
2:   if  $n \leq 1$  then
3:     return  $n$ 
4:   else
5:     return FIBRECURSIVO( $n - 1$ ) + FIBRECURSIVO( $n - 2$ )
6:   end if
7: end function
```

Observación algorítmica

La estructura recursiva directa genera un árbol de llamadas con recomputación masiva de subproblemas idénticos, conduciendo a una complejidad temporal exponencial $\Theta(\phi^n)$ donde $\phi = (1 + \sqrt{5})/2$ es el número áureo.

Ejemplo: sucesión de Fibonacci (derivación de la recurrencia) I

La recurrencia se deriva directamente de la semántica del algoritmo:

- ▶ **Caso base:** Para $n \leq 1$, la función retorna n inmediatamente, estableciendo las condiciones iniciales $F_0 = 0$ y $F_1 = 1$.
- ▶ **Caso recursivo:** Para $n \geq 2$, la función realiza dos llamadas recursivas y retorna su suma, lo que implica:

$$F_n = F_{n-1} + F_{n-2}.$$

La recurrencia resultante es lineal, homogénea, de segundo orden y con coeficientes constantes:

$$F_n - F_{n-1} - F_{n-2} = 0, \quad n \geq 2.$$

Ejemplo: sucesión de Fibonacci (ecuación característica) I

Se supone una solución de la forma $F_n = r^n$ con $r \neq 0$. Sustituyendo en la recurrencia:

$$r^n - r^{n-1} - r^{n-2} = 0.$$

Dividiendo por r^{n-2} :

$$r^2 - r - 1 = 0.$$

Esta es la ecuación característica asociada a la recurrencia de Fibonacci.

Ejemplo: sucesión de Fibonacci (raíces del polinomio) I

Las raíces de la ecuación característica $r^2 - r - 1 = 0$ son:

$$r = \frac{1 \pm \sqrt{5}}{2}.$$

Definimos las constantes fundamentales:

$$\phi = \frac{1 + \sqrt{5}}{2} \approx 1.61803, \quad \psi = \frac{1 - \sqrt{5}}{2} \approx -0.61803.$$

Observamos que ϕ es el número áureo y satisface $\phi^2 = \phi + 1$, mientras que $\psi = 1 - \phi = -1/\phi$.

Ejemplo: sucesión de Fibonacci (solución general) I

Como existen dos raíces reales y distintas, la solución general de la recurrencia homogénea es:

$$F_n = c_1\phi^n + c_2\psi^n.$$

Las constantes c_1 y c_2 se determinan mediante las condiciones iniciales.

Ejemplo: sucesión de Fibonacci (determinación de constantes) I

Aplicando las condiciones iniciales:

Para $n = 0$:

$$F_0 = 0 = c_1\phi^0 + c_2\psi^0 = c_1 + c_2.$$

Para $n = 1$:

$$F_1 = 1 = c_1\phi + c_2\psi.$$

El sistema resultante es:

$$\begin{cases} c_1 + c_2 = 0, \\ c_1\phi + c_2\psi = 1. \end{cases}$$

Resolviendo, $c_2 = -c_1$ y sustituyendo en la segunda ecuación:

$$c_1(\phi - \psi) = 1.$$

Ejemplo: sucesión de Fibonacci (determinación de constantes) II

Como $\phi - \psi = \sqrt{5}$, se obtiene:

$$c_1 = \frac{1}{\sqrt{5}}, \quad c_2 = -\frac{1}{\sqrt{5}}.$$

Ejemplo: sucesión de Fibonacci (fórmula cerrada de Binet) I

La fórmula cerrada para el n -ésimo número de Fibonacci es:

$$F_n = \frac{\phi^n - \psi^n}{\sqrt{5}} = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right].$$

Propiedad notable

A pesar de que ϕ y ψ son números irracionales, la combinación $\frac{\phi^n - \psi^n}{\sqrt{5}}$ produce siempre un número entero para todo $n \in \mathbb{N}$, debido a la cancelación exacta de las partes irracionales al considerar los conjugados algebraicos.

Ejemplo: sucesión de Fibonacci (comportamiento asintótico)

|

Como $|\psi| < 1$, el término ψ^n decrece exponencialmente hacia cero cuando $n \rightarrow \infty$. Por lo tanto:

$$\lim_{n \rightarrow \infty} \frac{F_n}{\phi^n / \sqrt{5}} = \lim_{n \rightarrow \infty} \left(1 - \left(\frac{\psi}{\phi} \right)^n \right) = 1,$$

pues $|\psi/\phi| = |\psi|/\phi < 1$.

Esto establece que:

$$F_n \sim \frac{\phi^n}{\sqrt{5}} \quad \text{cuando } n \rightarrow \infty.$$

En términos de notación asintótica:

$$F_n = \Theta(\phi^n).$$

Ejemplo: sucesión de Fibonacci (comportamiento asintótico)

II

Consecuencia algorítmica

La complejidad temporal del algoritmo recursivo directo para calcular F_n es $\Theta(\phi^n)$, lo que explica su ineficiencia práctica incluso para valores moderados de n . Este crecimiento exponencial contrasta drásticamente con la complejidad lineal $\Theta(n)$ de la versión iterativa o la complejidad logarítmica $\Theta(\log n)$ alcanzable mediante exponenciación matricial.

Caso 2: Recurrencias lineales no homogéneas

Recurrencias lineales no homogéneas: formulación general I

Definición formal

Una recurrencia lineal no homogénea de orden k con coeficientes constantes tiene la forma:

$$T(n) + a_1 T(n-1) + \cdots + a_k T(n-k) = f(n),$$

donde $a_1, \dots, a_k$ son constantes reales con $a_k \neq 0$, y $f(n)$ es una función no homogénea que típicamente adopta la forma:

$$f(n) = \sum_{i=1}^m b_i^n p_i(n),$$

siendo b_i constantes no nulas y $p_i(n)$ polinomios de grado $\deg(p_i)$.

Recurrencias lineales no homogéneas: formulación general II

Método de solución constructivo

La solución general se expresa como suma de dos componentes:

$$T(n) = T_h(n) + T_p(n),$$

donde $T_h(n)$ es la solución de la ecuación homogénea asociada y $T_p(n)$ es una solución particular de la ecuación no homogénea.

Recurrencias no homogéneas: determinación de la solución particular I

Forma de la solución particular

Para cada término forzante $b_i^n p_i(n)$ en $f(n)$:

- ▶ Si b_i no es raíz del polinomio característico homogéneo:

$$T_{p,i}(n) = q_i(n) b_i^n, \quad \deg(q_i) = \deg(p_i).$$

- ▶ Si b_i es raíz de multiplicidad $s \geq 1$:

$$T_{p,i}(n) = n^s q_i(n) b_i^n, \quad \deg(q_i) = \deg(p_i).$$

Recurrencias no homogéneas: determinación de la solución particular II

Ecuación característica extendida

Alternativamente, se construye el polinomio característico extendido:

$$\left(r^k + a_1 r^{k-1} + \dots + a_k\right) \prod_{i=1}^m (r - b_i)^{\deg(p_i)+1} = 0.$$

Si sus raíces son $r_1, \dots, r_\ell$ con multiplicidades $m_1, \dots, m_\ell$, la solución general es:

$$T(n) = \sum_{i=1}^{\ell} q_i(n) r_i^n,$$

donde cada $q_i(n)$ es un polinomio de grado menor que m_i .

Ejemplo: Torres de Hanoi (enunciado) I

Problema. Determinar el número mínimo de movimientos $T(n)$ necesarios para transferir n discos desde un poste origen a un poste destino utilizando un poste auxiliar, respetando las reglas clásicas del rompecabezas:

- ▶ Solo se puede mover un disco a la vez.
- ▶ Un disco solo puede colocarse sobre un disco de mayor tamaño o sobre un poste vacío.

Derive la ecuación de recurrencia que modela $T(n)$ y determine su solución cerrada mediante el método de ecuaciones características.

Ejemplo: Torres de Hanoi (algoritmo recursivo)

Ejemplo: Torres de Hanoi (algoritmo recursivo) I

Algoritmo 14 Hanoi(n , origen, destino, auxiliar)

```
1: function HANOI( $n$ , origen, destino, auxiliar)
2:   if  $n = 1$  then
3:     Mover disco de origen a destino
4:   else
5:     HANOI( $n - 1$ , origen, auxiliar, destino)
6:     Mover disco de origen a destino
7:     HANOI( $n - 1$ , auxiliar, destino, origen)
8:   end if
9: end function
```

Ejemplo: Torres de Hanoi (algoritmo recursivo) II

Observación estructural

El algoritmo descompone el problema de tamaño n en dos subproblemas de tamaño $n - 1$ separados por una operación elemental (movimiento del disco base), lo que induce directamente una recurrencia lineal no homogénea de primer orden.

Ejemplo: Torres de Hanoi (derivación de la recurrencia) I

Análisis por casos

- ▶ **Caso base** ($n = 1$): Se requiere exactamente un movimiento:

$$T(1) = 1.$$

- ▶ **Caso recursivo** ($n > 1$): La solución requiere:

1. Mover $n - 1$ discos del origen al auxiliar: $T(n - 1)$ movimientos.
2. Mover el disco base del origen al destino: 1 movimiento.
3. Mover $n - 1$ discos del auxiliar al destino: $T(n - 1)$ movimientos.

Por lo tanto:

$$T(n) = 2T(n - 1) + 1.$$

Ejemplo: Torres de Hanoi (derivación de la recurrencia) II

La recurrencia completa es:

$$T(n) = \begin{cases} 1 & \text{si } n = 1, \\ 2T(n-1) + 1 & \text{si } n > 1. \end{cases}$$

Esta es una recurrencia lineal no homogénea de primer orden con coeficientes constantes.

Ejemplo: Torres de Hanoi (solución: método constructivo) I

Paso 1. Solución de la ecuación homogénea asociada

La ecuación homogénea es:

$$T_h(n) - 2T_h(n-1) = 0.$$

Suponiendo $T_h(n) = r^n$:

$$r^n - 2r^{n-1} = 0 \implies r = 2.$$

Por lo tanto:

$$T_h(n) = c_1 2^n.$$

Ejemplo: Torres de Hanoi (solución: método constructivo) II

Paso 2. Solución particular

La parte no homogénea es $f(n) = 1 = 1^n \cdot 1$, donde $b = 1$ y $p(n) = 1$ (grado 0). Como $b = 1$ no es raíz del polinomio homogéneo ($r = 2$), proponemos:

$$T_p(n) = c_2.$$

Sustituyendo en la recurrencia original:

$$c_2 = 2c_2 + 1 \implies c_2 = -1.$$

Por lo tanto:

$$T_p(n) = -1.$$

Ejemplo: Torres de Hanoi (solución general y constantes) I

Paso 3. Solución general

La solución general es la suma de las componentes homogénea y particular:

$$T(n) = T_h(n) + T_p(n) = c_1 2^n - 1.$$

Paso 4. Determinación de constantes

Utilizando la condición inicial $T(1) = 1$:

$$1 = c_1 \cdot 2^1 - 1 \implies 2c_1 = 2 \implies c_1 = 1.$$

Paso 5. Fórmula cerrada

La solución explícita es:

$$T(n) = 2^n - 1.$$

Verificación para valores pequeños:

$$T(1) = 2^1 - 1 = 1, \quad T(2) = 2^2 - 1 = 3, \quad T(3) = 2^3 - 1 = 7.$$

Ejemplo: Torres de Hanoi (solución: ecuación característica extendida) I

Paso 1. Polinomio característico extendido

Reescribimos la recurrencia como:

$$T(n) - 2T(n-1) = 1 = 1^n \cdot 1.$$

El polinomio característico extendido es:

$$(r-2)(r-1)^{0+1} = (r-2)(r-1) = 0.$$

Sus raíces son $r_1 = 2$ (multiplicidad 1) y $r_2 = 1$ (multiplicidad 1).

Paso 2. Forma de la solución general

Como ambas raíces tienen multiplicidad 1, la solución general es:

$$T(n) = c_1 2^n + c_2 1^n = c_1 2^n + c_2.$$

Ejemplo: Torres de Hanoi (solución: ecuación característica extendida) II

Paso 3. Determinación de constantes

Utilizando $T(1) = 1$ y calculando $T(2) = 2T(1) + 1 = 3$:

$$\begin{cases} 2c_1 + c_2 = 1, \\ 4c_1 + c_2 = 3. \end{cases}$$

Resolviendo: $c_1 = 1$, $c_2 = -1$.

Por lo tanto:

$$T(n) = 2^n - 1.$$

Ejemplo: Torres de Hanoi (comportamiento asintótico) I

Consideremos el cociente:

$$\frac{T(n)}{2^n} = 1 - \frac{1}{2^n}.$$

Para $n \geq 1$ se verifica:

$$\frac{1}{2} = \frac{T(1)}{2^1} \leq \frac{T(n)}{2^n} < 1,$$

pues la sucesión $\frac{T(n)}{2^n}$ es estrictamente creciente y converge a 1 cuando $n \rightarrow \infty$.

Por lo tanto, existen constantes $c_1 = \frac{1}{2}$ y $c_2 = 1$ tales que:

$$c_1 2^n \leq T(n) \leq c_2 2^n, \quad \forall n \geq 1.$$

Ejemplo: Torres de Hanoi (comportamiento asintótico) II

Esto establece rigurosamente que:

$$T(n) = \Theta(2^n).$$

Consecuencia algorítmica

El número mínimo de movimientos crece exponencialmente con el número de discos. Para $n = 64$ (versión clásica del problema), se requieren $2^{64} - 1 \approx 1.84 \times 10^{19}$ movimientos. Asumiendo un movimiento por segundo, el tiempo necesario superaría los 500 mil millones de años, ilustrando dramáticamente las implicaciones prácticas del crecimiento exponencial en algoritmos recursivos sin optimización.

Ejemplo completo: recurrencia no homogénea

Ejemplo: enunciado del problema I

Resolver la recurrencia lineal no homogénea de segundo orden:

$$T(n) - 3T(n-1) + 2T(n-2) = 5 \cdot 2^n + n, T(0) = 1 \text{ y } T(1) = 4.$$

Determine la solución cerrada explícita y su comportamiento asintótico utilizando el método de ecuaciones características.

Solución: método constructivo (parte homogénea) I

Paso 1. Ecuación homogénea asociada

La ecuación homogénea es:

$$T_h(n) - 3T_h(n-1) + 2T_h(n-2) = 0.$$

Paso 2. Polinomio característico

Suponiendo solución de la forma $T_h(n) = r^n$:

$$r^n - 3r^{n-1} + 2r^{n-2} = 0 \implies r^2 - 3r + 2 = 0.$$

Factorizando:

$$(r-1)(r-2) = 0.$$

Las raíces son $r_1 = 1$ y $r_2 = 2$, ambas simples.

Solución: método constructivo (parte homogénea) II

Paso 3. Solución homogénea

Como las raíces son reales y distintas:

$$T_h(n) = c_1 \cdot 1^n + c_2 \cdot 2^n = c_1 + c_2 2^n.$$

Solución: método constructivo (parte particular - primer término) I

Paso 4. Primer término no homogéneo: $5 \cdot 2^n$

El término $5 \cdot 2^n$ tiene la forma $b_1^n p_1(n)$ con $b_1 = 2$ y $p_1(n) = 5$ (grado 0). Como $b_1 = 2$ es raíz simple del polinomio característico homogéneo, la solución particular propuesta es:

$$T_{p,1}(n) = n^s q_1(n) b_1^n = n^1 \cdot c_3 \cdot 2^n = c_3 n 2^n,$$

donde $s = 1$ es la multiplicidad de la raíz $b_1 = 2$ y $\deg(q_1) = \deg(p_1) = 0$.

Solución: método constructivo (parte particular - segundo término) I

Paso 5. Segundo término no homogéneo: n

El término n tiene la forma $b_2^n p_2(n)$ con $b_2 = 1$ y $p_2(n) = n$ (grado 1).

Como $b_2 = 1$ es raíz simple del polinomio característico homogéneo, la solución particular propuesta es:

$$T_{p,2}(n) = n^s q_2(n) b_2^n = n^1 \cdot (c_4 n + c_5) \cdot 1^n = c_4 n^2 + c_5 n,$$

donde $s = 1$ es la multiplicidad de la raíz $b_2 = 1$ y $\deg(q_2) = \deg(p_2) = 1$.

Solución: método constructivo (parte particular - segundo término) II

Paso 6. Solución particular total

La solución particular completa es la suma de ambas contribuciones:

$$T_p(n) = T_{p,1}(n) + T_{p,2}(n) = c_3 n 2^n + c_4 n^2 + c_5 n.$$

Solución: método constructivo (determinación de coeficientes) I

Paso 7. Sustitución en la recurrencia original

Sustituyendo $T_p(n)$ en la ecuación no homogénea:

$$T_p(n) - 3T_p(n-1) + 2T_p(n-2) = 5 \cdot 2^n + n.$$

Desarrollando cada término y agrupando coeficientes de 2^n , n y términos constantes:

$$\frac{c_3}{2} 2^n - 2c_4 n + (5c_4 - c_5) = 5 \cdot 2^n + n.$$

Solución: método constructivo (determinación de coeficientes) II

Paso 8. Sistema de ecuaciones para los coeficientes

Igualando coeficientes de términos homólogos:

$$\begin{cases} \frac{c_3}{2} = 5, \\ -2c_4 = 1, \\ 5c_4 - c_5 = 0. \end{cases}$$

Resolviendo:

$$c_3 = 10, \quad c_4 = -\frac{1}{2}, \quad c_5 = -\frac{5}{2}.$$

Por lo tanto:

$$T_p(n) = 10 n 2^n - \frac{1}{2} n^2 - \frac{5}{2} n.$$

Solución: método constructivo (solución general y constantes) I

Paso 9. Solución general

La solución general es la suma de las componentes homogénea y particular:

$$T(n) = T_h(n) + T_p(n) = c_1 + c_2 2^n + 10 n 2^n - \frac{1}{2} n^2 - \frac{5}{2} n.$$

Paso 10. Determinación de constantes mediante condiciones iniciales

Aplicando $T(0) = 1$:

$$1 = c_1 + c_2 \cdot 1 + 0 - 0 - 0 \implies c_1 + c_2 = 1.$$

Solución: método constructivo (solución general y constantes) II

Aplicando $T(1) = 4$:

$$4 = c_1 + c_2 \cdot 2 + 10 \cdot 1 \cdot 2 - \frac{1}{2} - \frac{5}{2} \implies c_1 + 2c_2 = -13.$$

Resolviendo el sistema:

$$c_2 = -14, \quad c_1 = 15.$$

Solución: método constructivo (resultado final) I

Paso 11. Solución cerrada explícita

Sustituyendo las constantes determinadas:

$$T(n) = 15 + (-14 + 10n) 2^n - \frac{1}{2}n^2 - \frac{5}{2}n.$$

Solución: método constructivo (resultado final) II

Paso 12. Comportamiento asintótico

El término dominante cuando $n \rightarrow \infty$ es $10n2^n$, ya que crece exponencialmente multiplicado por un factor lineal, mientras que los demás términos son polinómicos o exponenciales de menor base.

Formalmente, consideremos el cociente:

$$\frac{T(n)}{n2^n} = \frac{15}{n2^n} - \frac{14}{n} + 10 - \frac{1}{2} \cdot \frac{n}{2^n} - \frac{5}{2} \cdot \frac{1}{2^n}.$$

Tomando límite cuando $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} \frac{T(n)}{n2^n} = 10.$$

Solución: método constructivo (resultado final) III

Por lo tanto, existen constantes $c_1, c_2 > 0$ y $n_0 \in \mathbb{N}$ tales que:

$$c_1 n 2^n \leq T(n) \leq c_2 n 2^n, \quad \forall n \geq n_0.$$

Esto establece rigurosamente que:

$$T(n) = \Theta(n 2^n).$$

Solución: ecuación característica extendida I

Paso 1. Polinomio característico extendido

El polinomio característico de la parte homogénea es:

$$r^2 - 3r + 2 = (r - 1)(r - 2).$$

Para el término no homogéneo $5 \cdot 2^n + n$:

- ▶ $5 \cdot 2^n$: $b_1 = 2$, $\deg(p_1) = 0 \implies$ factor $(r - 2)^{0+1} = (r - 2)$.
- ▶ $n = 1^n \cdot n$: $b_2 = 1$, $\deg(p_2) = 1 \implies$ factor $(r - 1)^{1+1} = (r - 1)^2$.

El polinomio característico extendido es:

$$(r - 1)(r - 2) \cdot (r - 2) \cdot (r - 1)^2 = (r - 1)^3(r - 2)^2 = 0.$$

Solución: ecuación característica extendida II

Paso 2. Raíces y multiplicidades

- ▶ $r = 1$ con multiplicidad $m_1 = 3$.
- ▶ $r = 2$ con multiplicidad $m_2 = 2$.

Solución: ecuación característica extendida (estructura y constantes) I

Paso 3. Forma de la solución general

La solución general tiene la forma:

$$T(n) = (c_{11} + c_{12}n) 2^n + (c_{21} + c_{22}n + c_{23}n^2) 1^n = (c_{11} + c_{12}n) 2^n + c_{21} + c_{22}n + c_{23}n^2$$

Solución: ecuación característica extendida (estructura y constantes) II

Paso 4. Determinación de constantes

Como el polinomio extendido es de grado 5, se requieren 5 condiciones. Utilizamos $T(0)$ y $T(1)$ dadas, y calculamos $T(2)$, $T(3)$ y $T(4)$ mediante la recurrencia:

$$T(2) = 3T(1) - 2T(0) + 5 \cdot 2^2 + 2 = 32,$$

$$T(3) = 3T(2) - 2T(1) + 5 \cdot 2^3 + 3 = 131,$$

$$T(4) = 3T(3) - 2T(2) + 5 \cdot 2^4 + 4 = 413.$$

Resolviendo el sistema lineal resultante se obtiene:

$$c_{11} = -14, \quad c_{12} = 10, \quad c_{21} = 15, \quad c_{22} = -\frac{5}{2}, \quad c_{23} = -\frac{1}{2}.$$

Solución: ecuación característica extendida (estructura y constantes) III

Paso 5. Solución cerrada

Sustituyendo las constantes:

$$T(n) = (-14 + 10n) 2^n + 15 - \frac{5}{2}n - \frac{1}{2}n^2,$$

coincidiendo exactamente con el resultado obtenido mediante el método constructivo.

Comparación de métodos y conclusión I

Ventajas del método constructivo

- ▶ Requiere resolver sistemas lineales de tamaño igual al orden de la recurrencia homogénea (en este caso, 2×2).
- ▶ Permite identificar explícitamente las contribuciones de cada término no homogéneo.
- ▶ Es más eficiente computacionalmente cuando el número de términos no homogéneos es pequeño.

Comparación de métodos y conclusión II

Ventajas del polinomio característico extendido

- ▶ Unifica el tratamiento de la parte homogénea y no homogénea en un solo polinomio.
- ▶ Automatiza la elección de la forma de la solución particular, evitando errores en casos de resonancia.
- ▶ Es especialmente útil cuando la parte no homogénea es una combinación compleja de exponenciales y polinomios.

Comparación de métodos y conclusión III

Conclusión

Ambos métodos son equivalentes y conducen a la misma solución cerrada. La elección entre ellos depende del contexto específico y de las preferencias metodológicas del analista. El comportamiento asintótico dominante $\Theta(n 2^n)$ refleja que el término $5 \cdot 2^n$ en la parte no homogénea, al coincidir con una raíz del polinomio característico, genera una solución particular con factor lineal adicional n , elevando el orden de crecimiento respecto a una solución puramente exponencial 2^n .

Alcances, existencia y limitaciones

Existencia, unicidad y alcance del método I

Teorema de existencia y unicidad

Sea una recurrencia lineal de orden k con coeficientes constantes:

$$T(n) + a_1T(n-1) + \cdots + a_kT(n-k) = f(n),$$

donde $a_1, \dots, a_k \in \mathbb{R}$ con $a_k \neq 0$, y $f(n)$ es una función dada. Entonces, para cualquier conjunto de k condiciones iniciales especificadas:

$$T(0) = t_0, T(1) = t_1, \dots, T(k-1) = t_{k-1},$$

existe una única función $T: \mathbb{N} \rightarrow \mathbb{R}$ que satisface simultáneamente la recurrencia y las condiciones iniciales.

Existencia, unicidad y alcance del método II

Dimensión del espacio de soluciones

El conjunto de todas las soluciones de la ecuación homogénea asociada forma un espacio vectorial de dimensión k sobre $\mathbb{R}$. Por lo tanto, la solución general de la parte homogénea depende de exactamente k constantes arbitrarias, las cuales se determinan unívocamente mediante las k condiciones iniciales.

Aplicaciones del método de ecuaciones características I

El método de ecuaciones características constituye una herramienta fundamental en diversos contextos matemáticos y computacionales:

- ▶ **Análisis de algoritmos:** Resolución de recurrencias que modelan la complejidad temporal y espacial de algoritmos recursivos (ej.: Fibonacci, Torres de Hanoi, divide y vencerás).
- ▶ **Sistemas dinámicos discretos:** Modelado de fenómenos evolutivos en tiempo discreto mediante ecuaciones en diferencias lineales.
- ▶ **Procesos estocásticos:** Análisis de cadenas de Markov con estructura lineal y cálculo de distribuciones estacionarias.
- ▶ **Teoría de señales:** Diseño y análisis de filtros digitales lineales invariantes en el tiempo (LTI).
- ▶ **Matemática financiera:** Modelado de flujos de caja y valuación de instrumentos con pagos periódicos estructurados.

Restricciones del método clásico

El método de ecuación característica requiere:

- ▶ **Linealidad:** no aplica a recurrencias no lineales (ej.: $T(n) = T(n-1)^2 + 1$).
- ▶ **Coeficientes constantes:** no cubre recurrencias con coeficientes variables (ej.: $T(n) = n \cdot T(n-1) + 1$).
- ▶ **Resonancia:** exige ajustes en la forma de la solución particular cuando el término forzante coincide con una raíz homogénea.
- ▶ **Algebra compleja:** recurrencias de alto orden generan sistemas lineales engorrosos, aunque siempre resolubles.

Limitaciones y extensiones II

Técnicas complementarias

Para abordar casos fuera del alcance del método clásico existen extensiones y alternativas:

- ▶ Transformada Z para recurrencias lineales con coeficientes constantes.
- ▶ Método de variación de parámetros para coeficientes variables.
- ▶ Análisis asintótico mediante teorema maestro o métodos de perturbación.
- ▶ Técnicas de generación de funciones para recurrencias complejas.

Teorema maestro

Teorema maestro: formulación rigurosa I

Enunciado formal

Sea $T: \mathbb{N} \rightarrow \mathbb{R}$ una función que satisface la recurrencia

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad n > 1,$$

donde $a \geq 1$ y $b > 1$ son constantes reales, y $f(n)$ es una función asintóticamente positiva que representa el coste de división y combinación. Defínase el exponente crítico

$$d = \log_b a.$$

El comportamiento asintótico de $T(n)$ se determina según la relación entre $f(n)$ y n^d , clasificándose en tres casos mutuamente excluyentes.

Teorema maestro: caso 1 (dominancia de las hojas)

Teorema maestro: caso 1 (dominancia de las hojas) I

Condición

Existe $\varepsilon > 0$ tal que

$$f(n) = O\left(n^{d-\varepsilon}\right).$$

Conclusión

El coste total está dominado por el trabajo realizado en las hojas del árbol de recursión:

$$T(n) = \Theta\left(n^d\right).$$

Teorema maestro: caso 1 (dominancia de las hojas) II

Interpretación algorítmica

El trabajo realizado en los niveles superiores del árbol de recursión es asintóticamente despreciable frente al acumulado en las hojas. Este escenario ocurre típicamente en algoritmos donde la división genera muchos subproblemas pequeños cuyo procesamiento individual es costoso.

Teorema maestro: caso 2 (equilibrio de costes)

Teorema maestro: caso 2 (equilibrio de costes) I

Condición generalizada

Existe $k \in \mathbb{R}$ tal que

$$f(n) = \Theta\left(n^d \log^k n\right).$$

Conclusión según el valor de k

$$T(n) = \begin{cases} \Theta(n^d) & \text{si } k < -1, \\ \Theta(n^d \log \log n) & \text{si } k = -1, \\ \Theta(n^d \log^{k+1} n) & \text{si } k > -1. \end{cases}$$

Teorema maestro: caso 2 (equilibrio de costes) II

Caso clásico

Cuando $k = 0$ (es decir, $f(n) = \Theta(n^d)$), se obtiene la forma tradicional:

$$T(n) = \Theta\left(n^d \log n\right).$$

Esta situación representa un equilibrio perfecto donde el trabajo se distribuye uniformemente entre todos los niveles del árbol de recursión.

Teorema maestro: caso 3 (dominancia de la raíz)

Teorema maestro: caso 3 (dominancia de la raíz) I

Condiciones

1. Existe $\varepsilon > 0$ tal que

$$f(n) = \Omega\left(n^{d+\varepsilon}\right).$$

2. **Condición de regularidad:** existen constantes $c < 1$ y $n_0 \in \mathbb{N}$ tales que

$$a f\left(\frac{n}{b}\right) \leq c f(n), \quad \forall n \geq n_0.$$

Conclusión

El coste total está dominado por el trabajo realizado en la raíz del árbol de recursión:

$$T(n) = \Theta(f(n)).$$

Teorema maestro: caso 3 (dominancia de la raíz) II

Interpretación algorítmica

El trabajo realizado en los niveles superiores del árbol crece tan rápidamente que eclipsa el acumulado en niveles inferiores. La condición de regularidad garantiza que esta dominancia se mantenga consistentemente en todos los niveles, evitando oscilaciones patológicas en $f(n)$.

Teorema maestro: observaciones críticas I

Requisitos esenciales

- ▶ La brecha ε debe ser estrictamente positiva en los Casos 1 y 3; relaciones del tipo $f(n) = \Theta(n^d \log n)$ no satisfacen el Caso 1.
- ▶ La condición de regularidad en el Caso 3 es indispensable. Existen funciones $f(n)$ que cumplen $f(n) = \Omega(n^{d+\varepsilon})$ pero violan la condición de regularidad, conduciendo a comportamientos asintóticos no cubiertos por el teorema.
- ▶ El teorema presupone que n/b es entero o que T está definida para argumentos no enteros mediante redondeo apropiado (piso o techo), lo cual no altera el comportamiento asintótico.

Teorema maestro: observaciones críticas II

Limitaciones y extensiones

- ▶ No aplica a recurrencias no divisibles exactamente (ej.: $T(n) = T(n/2) + T(n/3) + n$), que requieren el método de Akra-Bazzi.
- ▶ Casos frontera donde $f(n)$ oscila entre los regímenes asintóticos (ej.: $f(n) = n^d \log \log n$) pueden requerir análisis ad hoc.
- ▶ Versiones extendidas del teorema incorporan factores logarítmicos en el Caso 2, como se presentó anteriormente, ampliando su aplicabilidad a algoritmos con costes de combinación no polinómicos puros.

Árbol de recursión: fundamentos conceptuales

Árbol de recursión: definición y propósito I

Concepto fundamental

El árbol de recursión es una representación gráfica que modela la estructura de llamadas de un algoritmo recursivo del tipo divide y vencerás. Cada nodo representa una invocación con un tamaño de subproblema específico, y las aristas representan las dependencias entre llamadas.

Árbol de recursión: definición y propósito II

Objetivo metodológico

El árbol permite visualizar y cuantificar sistemáticamente:

- ▶ Profundidad del árbol (número de niveles de recursión).
- ▶ Número de subproblemas en cada nivel.
- ▶ Tamaño de cada subproblema según el nivel.
- ▶ Coste de procesamiento por nodo (división y combinación).
- ▶ Distribución acumulada del trabajo computacional.

Esta representación constituye la base intuitiva para el teorema maestro y el análisis asintótico de algoritmos recursivos.

Estructura formal del árbol de recursión I

Consideremos la recurrencia canónica:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad a \geq 1, b > 1.$$

Características estructurales

- ▶ **Raíz (nivel 0):** problema original de tamaño n con coste $f(n)$.
- ▶ **Nivel i :** contiene a^i nodos, cada uno con subproblema de tamaño n/b^i y coste $f(n/b^i)$.
- ▶ **Profundidad ℓ :** nivel donde los subproblemas alcanzan tamaño constante:

$$\frac{n}{b^\ell} = 1 \quad \implies \quad \ell = \log_b n.$$

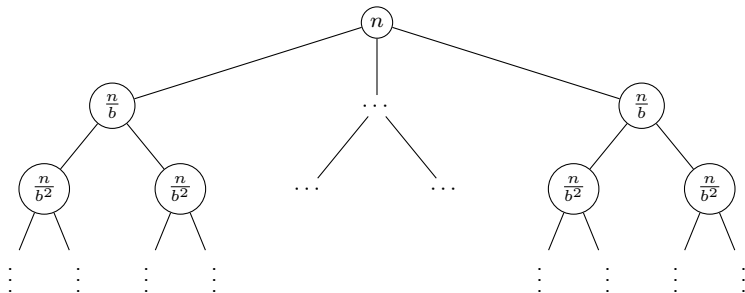
- ▶ **Hojas:** existen $a^\ell = n^{\log_b a}$ hojas, cada una con coste $\Theta(1)$.

Estructura formal del árbol de recursión II

El coste total es la suma del trabajo en todos los niveles:

$$T(n) = \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right) + \Theta\left(n^{\log_b a}\right).$$

Ilustración: estructura genérica del árbol I



Nivel 0: 1 nodo, coste $f(n)$

Nivel 1: a nodos, coste total $a \cdot f(n/b)$

Nivel 2: a^2 nodos, coste total $a^2 \cdot f(n/b^2)$

$\vdots$

Hojas: $a^{\log_b n} = n^{\log_b a}$ nodos, coste $\Theta(n^{\log_b a})$

Ilustración: estructura genérica del árbol II

Interpretación

Cada nivel i contribuye con $a^i \cdot f(n/b^i)$ al coste total. La suma de todas las contribuciones de los niveles internos más el coste de las hojas determina $T(n)$:

$$T(n) = \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right) + \Theta\left(n^{\log_b a}\right).$$

Ejemplo: merge sort I

Recurrencia:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad a = 2, \quad b = 2, \quad f(n) = \Theta(n).$$

Parámetros del árbol

- ▶ Exponente crítico: $d = \log_2 2 = 1$.
- ▶ Profundidad: $\ell = \log_2 n$.
- ▶ Número de hojas: $2^{\log_2 n} = n$.
- ▶ Coste por nivel i : $2^i \cdot \Theta(n/2^i) = \Theta(n)$ (constante en todos los niveles).

Ejemplo: merge sort II

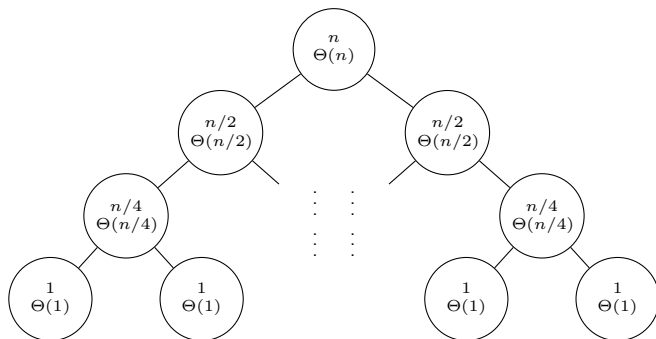
Distribución del trabajo

Como cada uno de los $\log_2 n + 1$ niveles contribuye con $\Theta(n)$:

$$T(n) = \Theta(n) \cdot (\log_2 n + 1) = \Theta(n \log n).$$

Esta es la situación del Caso 2 del teorema maestro ($f(n) = \Theta(n^d)$), donde el trabajo se distribuye uniformemente entre todos los niveles.

Ilustración: árbol de merge sort I



Nivel 0: $\Theta(n)$

Nivel 1: $2 \cdot \Theta(n/2) = \Theta(n)$

Nivel 2: $4 \cdot \Theta(n/4) = \Theta(n)$

$\vdots$

Nivel $\log_2 n$: $n \cdot \Theta(1) = \Theta(n)$

Ilustración: árbol de merge sort II

Conclusión

Cada nivel contribuye con exactamente $\Theta(n)$. Con $\log_2 n + 1$ niveles, el coste total es:

$$T(n) = \Theta(n) \cdot (\log_2 n + 1) = \Theta(n \log n).$$

Esta distribución uniforme ilustra el Caso 2 del teorema maestro ($f(n) = \Theta(n^{\log_b a})$ con $a = b = 2$).

Relación árbol-teorema maestro I

Caso 1: dominancia de las hojas

- ▶ Condición: $f(n) = O(n^{d-\varepsilon})$, $\varepsilon > 0$.
- ▶ Comportamiento: el coste por nivel decrece geométricamente al descender.
- ▶ Dominancia: trabajo en hojas $\Theta(n^d)$ supera a todos los niveles superiores.
- ▶ Ejemplo: $T(n) = 4T(n/2) + \Theta(n)$, $d = 2$, $f(n) = \Theta(n) = O(n^{2-\varepsilon})$.

Caso 2: equilibrio

- ▶ Condición: $f(n) = \Theta(n^d \log^k n)$.
- ▶ Comportamiento: coste por nivel aproximadamente constante.
- ▶ Dominancia: trabajo distribuido uniformemente entre niveles.
- ▶ Ejemplo: merge sort, $T(n) = 2T(n/2) + \Theta(n)$, $d = 1$.

Relación árbol-teorema maestro II

Caso 3: dominancia de la raíz

- ▶ Condición: $f(n) = \Omega(n^{d+\varepsilon})$ + regularidad.
- ▶ Comportamiento: coste por nivel decrece geométricamente al descender.
- ▶ Dominancia: trabajo en raíz $\Theta(f(n))$ supera a niveles inferiores.
- ▶ Ejemplo: $T(n) = T(n/2) + \Theta(n)$, $d = 0$, $f(n) = \Theta(n) = \Omega(n^\varepsilon)$.

Análisis cuantitativo del trabajo total I

El coste total se expresa como:

$$T(n) = \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right) + \Theta\left(n^{\log_b a}\right).$$

Comportamiento de la suma

La naturaleza de $\sum_{i=0}^{\log_b n - 1} a^i f(n/b^i)$ determina el caso:

- ▶ **Caso 1:** $f(n/b^i)$ decrece más rápido que $a^{-i} \Rightarrow$ serie converge, hojas dominan.
- ▶ **Caso 2:** $f(n/b^i) \approx a^{-i} n^d \Rightarrow \log_b n$ términos similares, coste $\Theta(n^d \log n)$.
- ▶ **Caso 3:** $f(n/b^i)$ decrece más lentamente que a^{-i} + regularidad $\Rightarrow$ primer término domina.

Análisis cuantitativo del trabajo total II

Esta perspectiva cuantitativa justifica formalmente la clasificación en tres casos y explica por qué la relación entre $f(n)$ y $n^{\log_b a}$ es el criterio determinante del comportamiento asintótico.

Ejemplos: Teorema maestro

Caso 1: dominancia de las hojas (enunciado) I

Ejemplo 1. Analice la recurrencia

$$T(n) = 2T\left(\frac{n}{2}\right) + n^{0.9}.$$

Determine su comportamiento asintótico aplicando el teorema maestro.

Caso 1: dominancia de las hojas (solución) I

Paso 1. Identificación de parámetros:

$$a = 2, \quad b = 2, \quad d = \log_b a = \log_2 2 = 1.$$

Paso 2. Comparación con n^d :

$$f(n) = n^{0.9} = n^{d-0.1}.$$

Existe $\varepsilon = 0.1 > 0$ tal que

$$f(n) = O\left(n^{d-\varepsilon}\right).$$

Conclusión. Por el Caso 1 del teorema maestro:

$$T(n) = \Theta\left(n^d\right) = \Theta(n).$$

Caso 1: dominancia de las hojas (enunciado) I

Ejemplo 2. Analice la recurrencia

$$T(n) = 3T\left(\frac{n}{3}\right) + (\log n)^2.$$

Determine su comportamiento asintótico aplicando el teorema maestro.

Caso 1: dominancia de las hojas (solución) I

Paso 1. Identificación de parámetros:

$$a = 3, \quad b = 3, \quad d = \log_b a = \log_3 3 = 1.$$

Paso 2. Comparación con n^d :

Para cualquier $\alpha, \beta > 0$ se verifica

$$\lim_{n \rightarrow \infty} \frac{(\log n)^\alpha}{n^\beta} = 0,$$

pues aplicando L'Hôpital reiteradamente el numerador crece más lentamente que cualquier potencia positiva de n .

En particular, con $\varepsilon = 1/2$:

$$\lim_{n \rightarrow \infty} \frac{(\log n)^2}{n^{1/2}} = 0 \quad \implies \quad f(n) = O\left(n^{1/2}\right) = O\left(n^{d-\varepsilon}\right).$$

Caso 1: dominancia de las hojas (solución) II

Conclusión. Por el Caso 1 del teorema maestro:

$$T(n) = \Theta\left(n^d\right) = \Theta(n).$$

Caso 1: dominancia de las hojas (enunciado) I

Ejemplo 3. Analice la recurrencia

$$T(n) = 4T\left(\frac{n}{2}\right) + \exp\left(\sqrt{\log n}\right).$$

Determine su comportamiento asintótico aplicando el teorema maestro.

Caso 1: dominancia de las hojas (solución) I

Paso 1. Identificación de parámetros:

$$a = 4, \quad b = 2, \quad d = \log_b a = \log_2 4 = 2.$$

Paso 2. Comparación con n^d :

Con $\varepsilon = 1$:

$$\frac{f(n)}{n^{d-\varepsilon}} = \frac{\exp(\sqrt{\log n})}{n} = \exp\left(\sqrt{\log n} - \log n\right) \xrightarrow{n \rightarrow \infty} 0,$$

pues $\log n$ crece asintóticamente más rápido que $\sqrt{\log n}$.

Por lo tanto,

$$f(n) = O\left(n^{d-\varepsilon}\right) = O(n).$$

Conclusión. Por el Caso 1 del teorema maestro:

$$T(n) = \Theta\left(n^d\right) = \Theta(n^2).$$

Caso 2: equilibrio de costes (enunciado) I

Ejemplo 1. Analice la recurrencia

$$T(n) = 2T\left(\frac{n}{2}\right) + n.$$

Determine su comportamiento asintótico aplicando el teorema maestro.

Caso 2: equilibrio de costes (solución) I

Paso 1. Identificación de parámetros:

$$a = 2, \quad b = 2, \quad d = \log_b a = \log_2 2 = 1.$$

Paso 2. Comparación con n^d :

$$f(n) = n = \Theta\left(n^d \log^0 n\right), \quad k = 0.$$

Conclusión. Por el Caso 2 del teorema maestro con $k = 0 > -1$:

$$T(n) = \Theta\left(n^d \log^{k+1} n\right) = \Theta(n \log n).$$

Observación algorítmica

Esta recurrencia modela el algoritmo de ordenamiento por mezcla (merge sort), cuya complejidad $\Theta(n \log n)$ representa el óptimo teórico para algoritmos de ordenamiento basados en comparaciones.

Caso 2: equilibrio de costes (enunciado) I

Ejemplo 2. Analice la recurrencia

$$T(n) = 2T\left(\frac{n}{2}\right) + n\left(\frac{1}{\log(n+e)} + 1\right).$$

Determine su comportamiento asintótico aplicando el teorema maestro.

Caso 2: equilibrio de costes (solución) I

Paso 1. Identificación de parámetros:

$$a = 2, \quad b = 2, \quad d = \log_b a = 1.$$

Paso 2. Acotación de $f(n)$:

Para $n \geq 1$ se verifica $1 \leq \log(n + e) \leq n + e$, de donde

$$1 \leq \frac{1}{\log(n + e)} + 1 \leq 2.$$

Por lo tanto,

$$n \leq f(n) \leq 2n \implies f(n) = \Theta(n) = \Theta\left(n^d \log^0 n\right), \quad k = 0.$$

Conclusión. Por el Caso 2 del teorema maestro:

$$T(n) = \Theta\left(n^d \log^{k+1} n\right) = \Theta(n \log n).$$

Caso 3: dominancia de la raíz (enunciado) I

Ejemplo 1. Analice la recurrencia

$$T(n) = 2T\left(\frac{n}{2}\right) + n^{1.1}.$$

Determine su comportamiento asintótico aplicando el teorema maestro, verificando explícitamente la condición de regularidad.

Caso 3: dominancia de la raíz (solución) I

Paso 1. Identificación de parámetros:

$$a = 2, \quad b = 2, \quad d = \log_b a = 1.$$

Paso 2. Verificación de la cota inferior:

$$f(n) = n^{1.1} = \Omega\left(n^{d+\varepsilon}\right) \quad \text{con } \varepsilon = 0.1 > 0.$$

Paso 3. Condición de regularidad:

$$a f\left(\frac{n}{b}\right) = 2 \left(\frac{n}{2}\right)^{1.1} = 2^{1-1.1} n^{1.1} = 2^{-0.1} n^{1.1} \leq c f(n),$$

donde $c = 2^{-0.1} \approx 0.933 < 1$.

Conclusión. Por el Caso 3 del teorema maestro:

$$T(n) = \Theta(f(n)) = \Theta(n^{1.1}).$$

Caso 3: dominancia de la raíz (enunciado) I

Ejemplo 2. Analice la recurrencia

$$T(n) = T\left(\frac{n}{2}\right) + n \log n.$$

Determine su comportamiento asintótico aplicando el teorema maestro.

Caso 3: dominancia de la raíz (solución) I

Paso 1. Identificación de parámetros:

$$a = 1, \quad b = 2, \quad d = \log_b a = \log_2 1 = 0.$$

Paso 2. Verificación de la cota inferior:

Con $\varepsilon = 1/2$:

$$\frac{f(n)}{n^{d+\varepsilon}} = \frac{n \log n}{n^{1/2}} = n^{1/2} \log n \xrightarrow{n \rightarrow \infty} \infty,$$

luego $f(n) = \Omega(n^{d+\varepsilon}) = \Omega(n^{1/2})$.

Paso 3. Condición de regularidad:

$$a f\left(\frac{n}{b}\right) = 1 \cdot \frac{n}{2} \log\left(\frac{n}{2}\right) = \frac{n}{2} (\log n - \log 2) \leq \frac{n}{2} \log n = \frac{1}{2} f(n),$$

donde $c = 1/2 < 1$.

Caso 3: dominancia de la raíz (solución) II

Conclusión. Por el Caso 3 del teorema maestro:

$$T(n) = \Theta(f(n)) = \Theta(n \log n).$$

Ejemplos clásicos

Karatsuba: multiplicación rápida de enteros (enunciado) I

Problema

Multiplicar dos enteros A y B de n bits mediante un algoritmo más eficiente que el método escolar cuadrático.

Idea algorítmica. Dividir cada entero en mitades de tamaño $n/2$, realizar tres multiplicaciones recursivas (en lugar de cuatro) y combinar los resultados mediante sumas y desplazamientos lineales.

Determine la complejidad temporal asintótica del algoritmo mediante el teorema maestro.

Karatsuba: algoritmo I

Algoritmo 15 Karatsuba(A, B, n)

```
1: function KARATSUBA( $A, B, n$ )
2:   if  $n \leq n_0$  then    //  $n_0$ : umbral para método escolar
3:     return  $A \times B$     // Multiplicación directa
4:   end if
5:    $m \leftarrow \lceil n/2 \rceil$ 
6:    $(A_1, A_0) \leftarrow \text{dividir}(A, m)$     //  $A = A_1 2^m + A_0$ 
7:    $(B_1, B_0) \leftarrow \text{dividir}(B, m)$     //  $B = B_1 2^m + B_0$ 
8:    $z_0 \leftarrow \text{KARATSUBA}(A_0, B_0, m)$ 
9:    $z_2 \leftarrow \text{KARATSUBA}(A_1, B_1, n - m)$ 
10:   $z_1 \leftarrow \text{KARATSUBA}(A_0 + A_1, B_0 + B_1, m + 1)$ 
11:   $z_1 \leftarrow z_1 - z_0 - z_2$ 
12:  return  $z_2 \cdot 2^{2m} + z_1 \cdot 2^m + z_0$ 
13: end function
```

Karatsuba: multiplicación rápida de enteros (solución) I

Recurrencia característica:

$$T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n).$$

Parámetros:

$$a = 3, \quad b = 2, \quad d = \log_b a = \log_2 3 \approx 1.585.$$

Clasificación:

$$f(n) = \Theta(n) = O\left(n^{d-\varepsilon}\right) \quad \text{con } \varepsilon = d - 1 > 0.$$

Por lo tanto, se aplica el Caso 1 del teorema maestro.

Solución:

$$T(n) = \Theta\left(n^{\log_2 3}\right).$$

Karatsuba: multiplicación rápida de enteros (solución) II

Observación histórica

El algoritmo de Karatsuba (1960) fue el primer método de multiplicación asintóticamente más rápido que el algoritmo escolar $\Theta(n^2)$, demostrando que la multiplicación no requiere necesariamente tiempo cuadrático. Este resultado inauguró la teoría moderna de algoritmos de multiplicación rápida.

Strassen: multiplicación rápida de matrices (enunciado) I

Problema

Multiplicar dos matrices $A, B \in \mathbb{R}^{n \times n}$ (con n potencia de 2) mediante un algoritmo más eficiente que el método cúbico estándar.

Idea algorítmica. Particionar cada matriz en cuatro cuadrantes de tamaño $n/2 \times n/2$, calcular siete productos recursivos $M_1, \dots, M_7$ mediante combinaciones lineales de los cuadrantes, y recombinar dichos productos mediante sumas y restas para obtener los cuadrantes de la matriz resultado.

Determine la complejidad temporal asintótica del algoritmo mediante el teorema maestro.

Strassen: algoritmo I

Algoritmo 16 Strassen(A, B, n)

- ```
1: function STRASSEN(A, B, n)
2: if $n \leq n_0$ then
3: return $A \times B$
4: end if
5: Particionar A y B en cuadrantes:
```

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

- ```
6:    $M_1 \leftarrow \text{STRASSEN}(A_{11} + A_{22}, B_{11} + B_{22}, n/2)$ 
7:    $M_2 \leftarrow \text{STRASSEN}(A_{21} + A_{22}, B_{11}, n/2)$ 
8:    $M_3 \leftarrow \text{STRASSEN}(A_{11}, B_{12} - B_{22}, n/2)$ 
9:    $M_4 \leftarrow \text{STRASSEN}(A_{22}, B_{21} - B_{11}, n/2)$ 
10:   $M_5 \leftarrow \text{STRASSEN}(A_{11} + A_{12}, B_{22}, n/2)$ 
11:   $M_6 \leftarrow \text{STRASSEN}(A_{21} - A_{11}, B_{11} + B_{12}, n/2)$ 
12:   $M_7 \leftarrow \text{STRASSEN}(A_{12} - A_{22}, B_{21} + B_{22}, n/2)$ 
13:  Calcular cuadrantes de  $C$ :
```

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

- ```
14: Combinar $C_{11}, C_{12}, C_{21}, C_{22}$ en C
15: return C
16: end function
```
-



# Strassen: multiplicación rápida de matrices (solución) I

**Recurrencia característica:**

$$T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2).$$

**Parámetros:**

$$a = 7, \quad b = 2, \quad d = \log_b a = \log_2 7 \approx 2.807.$$

**Clasificación:**

$$f(n) = \Theta(n^2) = O\left(n^{d-\varepsilon}\right) \quad \text{con } \varepsilon = d - 2 > 0.$$

Por lo tanto, se aplica el Caso 1 del teorema maestro.

**Solución:**

$$T(n) = \Theta\left(n^{\log_2 7}\right).$$

# Strassen: multiplicación rápida de matrices (solución) II

## Observación práctica

Aunque el algoritmo de Strassen (1969) presenta mejor complejidad asintótica que el método cúbico  $\Theta(n^3)$ , las constantes ocultas son significativamente mayores. En la práctica, solo supera al algoritmo estándar para matrices de tamaño considerable ( $n \gtrsim 100$ ), y presenta desventajas en estabilidad numérica y consumo de memoria adicional.

# Merge sort: ordenamiento por mezcla (enunciado) I

## Problema

Ordenar un arreglo  $A[1 \dots n]$  de  $n$  elementos en orden no decreciente.

**Idea algorítmica.** Aplicar la técnica de divide y vencerás:

1. Dividir el arreglo en dos mitades de tamaño aproximadamente igual.
2. Ordenar recursivamente cada mitad.
3. Fusionar las dos mitades ordenadas en un único arreglo ordenado mediante un procedimiento lineal.

*Determine la complejidad temporal asintótica del algoritmo mediante el teorema maestro.*

# Merge sort: algoritmo I

---

## Algoritmo 17 MergeSort( $A, p, r$ )

---

```
1: function MERGESORT(A, p, r)
2: if $p < r$ then
3: $q \leftarrow \lfloor (p + r) / 2 \rfloor$
4: MERGESORT(A, p, q)
5: MERGESORT($A, q + 1, r$)
6: MERGE(A, p, q, r)
7: end if
8: return A
9: end function
```

---

# Merge sort: algoritmo II

---

## Algoritmo 18 Merge( $A, p, q, r$ )

---

```
1: function MERGE(A, p, q, r)
2: $n_1 \leftarrow q - p + 1, \quad n_2 \leftarrow r - q$
3: Crear arreglos $L[1 \dots n_1 + 1], R[1 \dots n_2 + 1]$
4: for $i = 1$ to n_1 do
5: $L[i] \leftarrow A[p + i - 1]$
6: end for
7: for $j = 1$ to n_2 do
8: $R[j] \leftarrow A[q + j]$
9: end for
10: $L[n_1 + 1] \leftarrow \infty, \quad R[n_2 + 1] \leftarrow \infty$ //Centinelas
11: $i \leftarrow 1, \quad j \leftarrow 1$
12: for $k = p$ to r do
13: if $L[i] \leq R[j]$ then
14: $A[k] \leftarrow L[i]$
15: $i \leftarrow i + 1$
16: else
17: $A[k] \leftarrow R[j]$
18: $j \leftarrow j + 1$
19: end if
20: end for
21: end function
```

---

# Merge sort: ordenamiento por mezcla (solución) I

**Recurrencia característica:**

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n).$$

**Parámetros:**

$$a = 2, \quad b = 2, \quad d = \log_b a = \log_2 2 = 1.$$

**Clasificación:**

$$f(n) = \Theta(n) = \Theta\left(n^d \log^0 n\right), \quad k = 0.$$

Por lo tanto, se aplica el Caso 2 del teorema maestro con  $k = 0 > -1$ .

**Solución:**

$$T(n) = \Theta\left(n^d \log^{k+1} n\right) = \Theta(n \log n).$$

# Merge sort: ordenamiento por mezcla (solución) II

## Observación teórica

El merge sort alcanza la cota inferior  $\Omega(n \log n)$  para algoritmos de ordenamiento basados exclusivamente en comparaciones, siendo por tanto óptimo en el modelo de comparación. Su principal desventaja es el consumo espacial adicional  $\Theta(n)$  requerido para la operación de mezcla.

# Búsqueda binaria: localización en arreglo ordenado (enunciado) I

## Problema

Localizar la posición de un elemento  $x$  en un arreglo ordenado  $A[1 \dots n]$  de  $n$  elementos.

**Idea algorítmica.** Comparar  $x$  con el elemento central del intervalo de búsqueda actual; si coincide se retorna su posición; en caso contrario, se descarta la mitad del intervalo que no puede contener a  $x$  y se repite el proceso recursivamente sobre la mitad restante. El coste por nivel de recursión es constante  $\Theta(1)$ .

*Determine la complejidad temporal asintótica del algoritmo mediante el teorema maestro.*



# Búsqueda binaria: algoritmo iterativo

---

## Algoritmo 19 BinarySearch( $A, p, r, x$ )

---

```
1: function BINARYSEARCH(A, p, r, x)
2: while $p \leq r$ do
3: $m \leftarrow \lfloor (p + r)/2 \rfloor$
4: if $A[m] = x$ then
5: return m
6: else if $A[m] < x$ then
7: $p \leftarrow m + 1$
8: else
9: $r \leftarrow m - 1$
10: end if
11: end while
12: return -1 //Elemento no encontrado
13: end function
```

---

# Búsqueda binaria: localización en arreglo ordenado (solución) I

**Recurrencia característica:**

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1).$$

**Parámetros:**

$$a = 1, \quad b = 2, \quad d = \log_b a = \log_2 1 = 0.$$

**Clasificación:**

$$f(n) = \Theta(1) = \Theta(n^0 \log^0 n) = \Theta(n^d \log^0 n), \quad k = 0.$$

Por lo tanto, se aplica el Caso 2 del teorema maestro con  $k = 0 > -1$ .

# Búsqueda binaria: localización en arreglo ordenado (solución) II

**Solución:**

$$T(n) = \Theta\left(n^d \log^{k+1} n\right) = \Theta(\log n).$$

## Observación práctica

La búsqueda binaria representa el estándar de eficiencia para búsquedas en estructuras estáticas ordenadas. Su complejidad logarítmica implica que incluso para arreglos de tamaño astronómico ( $n \approx 10^{18}$ ), se requieren menos de 60 comparaciones en el peor caso. Esta propiedad la hace fundamental en sistemas de bases de datos y estructuras de datos como árboles binarios de búsqueda balanceados.

# Quickselect: selección del $k$ -ésimo menor (enunciado) I

## Problema

Encontrar el  $k$ -ésimo elemento más pequeño en un arreglo no ordenado de  $n$  elementos (por ejemplo, la mediana cuando  $k = \lfloor n/2 \rfloor$ ).

**Idea algorítmica.** Adaptar la estrategia de quicksort: particionar el arreglo respecto a un pivote y recursivamente procesar únicamente el subarreglo que contiene al elemento buscado (descartando el otro).

*Determine las complejidades temporal en el caso promedio y peor caso.*

# Quickselect: algoritmo

---

## Algoritmo 20 QuickSelect( $A$ , $izq$ , $der$ , $k$ )

---

```
1: function QUICKSELECT(A , izq , der , k)
2: if $izq = der$ then
3: return $A[izq]$
4: end if
5: $p \leftarrow$ PARTICIONAR(A , izq , der)
6: $rango \leftarrow p - izq + 1$ //Posición de $A[p]$ en subarreglo
7: if $k = rango$ then
8: return $A[p]$
9: else if $k < rango$ then
10: return QUICKSELECT(A , izq , $p - 1$, k)
11: else
12: return QUICKSELECT(A , $p + 1$, der , $k - rango$)
13: end if
14: end function
```

---

# Quickselect: análisis de complejidad I

## Peor caso

Particiones altamente desbalanceadas (pivote mínimo/máximo sistemáticamente):

$$T_{\text{peor}}(n) = T(n - 1) + \Theta(n) = \Theta(n^2).$$

# Quickselect: análisis de complejidad II

## Caso promedio

Bajo permutaciones aleatorias, el pivote divide el arreglo en proporciones esperadas constantes. La recurrencia esperada es:

$$T_{\text{prom}}(n) = \frac{1}{n} \sum_{i=1}^n T(\max\{i-1, n-i\}) + \Theta(n) = \Theta(n).$$

**Resultado fundamental:** Quickselect encuentra la mediana en tiempo lineal esperado, mientras que el mejor algoritmo de ordenamiento requiere  $\Omega(n \log n)$  comparaciones.

# Quickselect: análisis de complejidad III

## Observación práctica

La versión determinista *median-of-medians* garantiza  $\Theta(n)$  en el peor caso, pero sus constantes ocultas son tan grandes que en la práctica se prefiere la versión aleatorizada con complejidad esperada lineal.



# Integración teoría-práctica en el análisis de algoritmos

# Brecha entre análisis teórico e implementación práctica I

## Mismos algoritmos, diferentes comportamientos

Implementaciones distintas del mismo algoritmo pueden exhibir diferencias significativas en rendimiento real debido a:

- ▶ Estructuras de datos subyacentes (listas enlazadas vs. arreglos contiguos).
- ▶ Paradigma de ejecución (recursión vs. iteración con gestión explícita de pila).
- ▶ Interacción con la jerarquía de memoria (localidad espacial y temporal).
- ▶ Optimizaciones del compilador/intérprete (inlining, vectorización, eliminación de código muerto).

# Brecha entre análisis teórico e implementación práctica II

## Recomendación metodológica fundamental

El análisis asintótico proporciona una guía esencial para el diseño algorítmico, pero debe complementarse sistemáticamente con mediciones empíricas en escenarios representativos del dominio de aplicación. La teoría identifica tendencias asintóticas; la práctica revela el impacto de constantes ocultas, efectos de caché y características arquitectónicas específicas.

# Jerarquía práctica de funciones de complejidad I

## Orden estricto de crecimiento asintótico

$$\Theta(1) < \Theta(\log n) < \Theta(\sqrt{n}) < \Theta(n) < \Theta(n \log n) < \Theta(n^2) < \Theta(2^n) < \Theta(n!)$$

Esta jerarquía establece que, para entradas suficientemente grandes, cualquier algoritmo con complejidad en una clase inferior escalará mejor que uno en una clase superior, independientemente de las constantes multiplicativas.

# Jerarquía práctica de funciones de complejidad II

## Visualización y validación empírica

- ▶ Herramientas recomendadas: Python (matplotlib), GeoGebra, Desmos.
- ▶ Estrategia: representar múltiples funciones en escalas logarítmicas para observar puntos de cruce y dominancia asintótica.
- ▶ Aplicación práctica: determinar umbrales de tamaño de entrada donde un algoritmo teóricamente superior supera efectivamente a uno inferior (ej.:  $n \log n$  vs.  $n^2$  para  $n > 50$ ).

# Metodología rigurosa para medición empírica I

## Herramientas especializadas

- ▶ **Python:** `time.perf_counter` (alta resolución), `timeit` (micro-benchmarks), `cProfile` (análisis de llamadas), `memory_profiler` (consumo de memoria).
- ▶ **Sistemas Unix:** `/usr/bin/time -v` (métricas de sistema), `perf stat` (contadores de hardware), `hyperfine` (benchmarking estadístico robusto).
- ▶ **Entornos integrados:** extensiones de profiling en VS Code, PyCharm o Jupyter para análisis interactivo.

# Metodología rigurosa para medición empírica II

## Protocolo experimental reproducible

- ▶ Aislar estrictamente el código bajo análisis (excluir E/S, generación de datos y validación).
- ▶ Controlar el entorno: fijar semilla aleatoria, cerrar procesos en segundo plano, establecer afinidad de CPU, desactivar turbo-boost.
- ▶ Realizar calentamiento previo y múltiples réplicas independientes (mínimo 30 ejecuciones por tamaño de entrada).
- ▶ Utilizar escalas geométricas de  $n$  (ej.:  $n = 10^3, 10^4, \dots, 10^7$ ) para observar el régimen asintótico.
- ▶ Reportar estadísticos robustos: mediana y rango intercuartílico en lugar de media y desviación típica.

# Errores metodológicos frecuentes y su mitigación I

## Pitfalls en la medición

- ▶ Evaluar únicamente tamaños de entrada pequeños donde las constantes dominan el comportamiento asintótico.
- ▶ Incluir operaciones de E/S o inicialización en las mediciones de tiempo.
- ▶ Ignorar la variabilidad inherente del sistema operativo y hardware (sin réplicas ni estadísticos).
- ▶ No controlar la aleatoriedad en algoritmos probabilísticos (semillas no fijadas).



# Errores metodológicos frecuentes y su mitigación II

## Malinterpretaciones conceptuales

- ▶ Confundir  $O(\cdot)$  (cota superior asintótica) con tiempo de ejecución absoluto en una máquina específica.
- ▶ Asumir que  $O(n \log n)$  siempre supera a  $O(n^2)$  sin considerar constantes y efectos de caché para  $n$  moderados.
- ▶ Ignorar la distribución de las entradas: el caso promedio puede diferir drásticamente del peor caso.
- ▶ Omitir el tamaño de la salida en problemas de generación (el tiempo mínimo es  $\Omega(\text{tamaño de salida})$ ).

# Errores metodológicos frecuentes y su mitigación III

## Conclusión metodológica

El análisis asintótico constituye una herramienta teórica poderosa para la clasificación y diseño de algoritmos, pero su aplicación responsable exige:

1. Validación empírica rigurosa mediante protocolos experimentales reproducibles.
2. Análisis crítico de las discrepancias entre predicción teórica y comportamiento observado.
3. Consideración explícita de constantes ocultas, arquitectura hardware y patrones de acceso a memoria.

Solo la integración sistemática de teoría y práctica permite tomar decisiones informadas sobre la selección de algoritmos en contextos reales.

# Clases fundamentales

# Clases fundamentales: definición de $P$

# Clases fundamentales: definición de $P$ I

## Clase $P$

Conjunto de problemas de decisión que pueden resolverse por una máquina de Turing determinista en tiempo polinomial. Formalmente:

$$P = \bigcup_{k \in \mathbb{N}} \text{DTIME}(n^k).$$

# Clases fundamentales: definición de $P$ II

## Características esenciales

- ▶ Existe un algoritmo determinista que decide cualquier instancia del problema en tiempo  $O(n^k)$  para alguna constante  $k$ .
- ▶ Los problemas en  $P$  se consideran *tratables* desde el punto de vista teórico, aunque constantes grandes pueden hacerlos imprácticos para tamaños moderados de entrada.
- ▶  $P$  es cerrado bajo composición: la composición de dos algoritmos polinomiales es polinomial.

# Clases fundamentales: definición de $NP$

# Clases fundamentales: definición de $NP$ I

## Definición 1: Máquinas no deterministas

$$NP = \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k).$$

Un problema pertenece a  $NP$  si existe una máquina de Turing no determinista que lo resuelve en tiempo polinomial. La no determinismo permite "adivinar" la solución correcta y verificarla eficientemente.



# Clases fundamentales: definición de $NP$ II

## Definición 2: Verificación determinista

Un lenguaje  $L$  pertenece a  $NP$  si existe un verificador determinista  $V$  y un polinomio  $p$  tales que:

$$x \in L \iff \exists y \text{ con } |y| \leq p(|x|) \text{ tal que } V(x, y) = 1,$$

donde  $V$  opera en tiempo polinomial en  $|x|$ . La cadena  $y$  es un *certificado* o *testigo* de pertenencia.

# Clases fundamentales: definición de $NP$ III

## Relación fundamental

$$P \subseteq NP.$$

El problema abierto más importante de la ciencia computacional es determinar si esta inclusión es estricta:

$$P \stackrel{?}{=} NP.$$

# Clases fundamentales: ejemplos representativos

# Clases fundamentales: ejemplos representativos I

## Problemas en $P$

- ▶ **Ordenamiento:** MergeSort resuelve el problema en  $\Theta(n \log n) \subset O(n^2)$ .
- ▶ **Búsqueda binaria:** Localiza un elemento en un arreglo ordenado en  $\Theta(\log n)$ .
- ▶ **Camino más corto (Dijkstra):** Encuentra caminos mínimos en grafos con pesos no negativos en  $O((V + E) \log V)$ .

# Clases fundamentales: ejemplos representativos II

## Problemas en $NP$ (y conjeturadamente no en $P$ )

- ▶ **SAT (Satisfacibilidad booleana):** Determinar si existe una asignación de valores que satisfaga una fórmula proposicional en forma normal conjuntiva.
- ▶ **Problema de la mochila (decisión):** Dados objetos con pesos y valores, ¿existe un subconjunto con peso total  $\leq W$  y valor total  $\geq V$ ?
- ▶ **Clique máximo (decisión):** Dado un grafo  $G$  y entero  $k$ , ¿existe un subconjunto de  $k$  vértices donde cada par está conectado?

# NP-completitud: definición formal I

## Problemas NP-duros (NP-hard)

Un problema  $B$  es *NP-duro* si todo problema  $A \in NP$  se reduce polinomialmente a  $B$ :

$$\forall A \in NP, A \leq_p B.$$

Los problemas NP-duros son al menos tan difíciles como los más difíciles de  $NP$ , pero no necesariamente pertenecen a  $NP$  (pueden ser problemas de optimización o de decisión fuera de  $NP$ ).

# NP-completitud: definición formal II

## Problemas NP-completos

Un problema  $B$  es *NP-completo* si:

1.  $B \in NP$ , y
2.  $B$  es NP-duro.

Los problemas NP-completos representan los problemas más difíciles dentro de  $NP$ . Si algún problema NP-completo pertenece a  $P$ , entonces  $P = NP$ .

# Reducciones polinomiales: definición I

## Definición formal

Sean  $A$  y  $B$  problemas de decisión. Decimos que  $A$  se reduce polinomialmente a  $B$  (notación  $A \leq_p B$ ) si existe una función  $\varphi$  computable en tiempo polinomial tal que:

$$\forall x, \quad x \in A \iff \varphi(x) \in B.$$

## Propiedades fundamentales

- ▶ **Transitividad:** Si  $A \leq_p B$  y  $B \leq_p C$ , entonces  $A \leq_p C$ .
- ▶ **Preservación de tractabilidad:** Si  $A \leq_p B$  y  $B \in P$ , entonces  $A \in P$ .
- ▶ **Preservación de NP-completitud:** Si  $A$  es NP-completo,  $A \leq_p B$  y  $B \in NP$ , entonces  $B$  es NP-completo.



# Reducciones polinomiales: definición II

## Importancia metodológica

Las reducciones polinomiales constituyen la herramienta esencial para demostrar NP-completitud. El teorema de Cook-Levin establece que SAT es NP-completo; a partir de allí, se demuestra la NP-completitud de otros problemas mediante reducciones desde SAT o desde problemas ya conocidos como NP-completos.

## Ejemplo clásico de reducción: enunciado I

**Problema.** Demostrar que el problema de la clique máxima es NP-completo mediante reducción desde 3-SAT.

**Definiciones:**

- ▶ **3-SAT:** Dada una fórmula booleana  $\Phi$  en forma normal conjuntiva donde cada cláusula contiene exactamente tres literales, determinar si existe una asignación de valores que satisfaga  $\Phi$ .
- ▶ **CLIQUE:** Dado un grafo no dirigido  $G = (V, E)$  y un entero  $k$ , determinar si  $G$  contiene una clique de tamaño al menos  $k$  (subconjunto de vértices donde cada par está conectado por una arista).

**Objetivo.** Construir una transformación polinomial  $\varphi$  que mapee instancias de 3-SAT a instancias de CLIQUE tal que:

$$\Phi \text{ es satisfactible} \iff \varphi(\Phi) = (G, k) \text{ tiene una clique de tamaño } k.$$

# Ejemplo clásico de reducción: construcción I

## Construcción del grafo $G$ :

Sea  $\Phi = C_1 \wedge C_2 \wedge \dots \wedge C_m$  una fórmula 3-CNF con  $m$  cláusulas. Construimos  $G = (V, E)$  como sigue:

- ▶ **Vértices:** Para cada literal  $\ell$  en cada cláusula  $C_j$ , creamos un vértice  $v_{j,\ell}$ . Así,  $|V| = 3m$ .
- ▶ **Aristas:** Conectamos dos vértices  $v_{j,\ell}$  y  $v_{j',\ell'}$  si y solo si:
  1.  $j \neq j'$  (pertenecen a cláusulas distintas), y
  2.  $\ell$  y  $\ell'$  no son contradictorios (es decir, no son  $x$  y  $\neg x$  para la misma variable  $x$ ).
- ▶ **Parámetro  $k$ :** Establecemos  $k = m$ .

Esta construcción es claramente computable en tiempo polinomial  $O(m^2)$ .

## Ejemplo clásico de reducción: corrección I

( $\Rightarrow$ ) Si  $\Phi$  es satisfactible, entonces  $G$  tiene una  $m$ -clique

Sea  $\tau$  una asignación satisfactoria. Para cada cláusula  $C_j$ , seleccionamos un literal  $\ell_j$  que  $\tau$  satisface. Consideremos el conjunto  $S = \{v_{j,\ell_j} \mid 1 \leq j \leq m\}$ .  
Por construcción:

- ▶  $|S| = m = k$ .
- ▶ Los vértices en  $S$  pertenecen a cláusulas distintas ( $j \neq j'$ ).
- ▶ Como  $\tau$  satisface simultáneamente todos los literales  $\ell_j$ , ningún par  $\ell_j, \ell_{j'}$  puede ser contradictorio.

Por lo tanto, todos los pares en  $S$  están conectados, y  $S$  es una  $m$ -clique en  $G$ .

## Ejemplo clásico de reducción: corrección II

( $\Leftarrow$ ) Si  $G$  tiene una  $m$ -clique, entonces  $\Phi$  es satisfactible

Sea  $S$  una clique de tamaño  $m$  en  $G$ . Como  $G$  tiene  $m$  grupos de vértices (uno por cláusula) y una clique no puede contener dos vértices del mismo grupo (no hay aristas intra-cláusula),  $S$  debe contener exactamente un vértice  $v_{j,\ell_j}$  por cláusula  $C_j$ .

Definimos una asignación  $\tau$  que satisface cada  $\ell_j$  (si  $\ell_j = x$ , asignamos  $x = \text{verdadero}$ ; si  $\ell_j = \neg x$ , asignamos  $x = \text{falso}$ ). Como  $S$  es una clique, ningún par  $\ell_j, \ell_{j'}$  es contradictorio, por lo que  $\tau$  está bien definida y satisface todas las cláusulas.

# Otras clases de complejidad relevantes I

## Clase coNP

Conjunto de lenguajes cuyos complementos pertenecen a  $NP$ :

$$\text{coNP} = \{L \mid \overline{L} \in NP\}.$$

Ejemplo: UNSAT (determinar si una fórmula booleana es insatisfactible) pertenece a coNP pero no se sabe si pertenece a  $NP$ . Se conjetura que  $NP \neq \text{coNP}$ .

# Otras clases de complejidad relevantes II

## Clase PSPACE

Problemas resolubles con espacio polinomial (sin restricción en el tiempo):

$$\text{PSPACE} = \bigcup_{k \in \mathbb{N}} \text{DSPACE}(n^k).$$

Se sabe que  $NP \subseteq \text{PSPACE}$ , y se conjetura que la inclusión es estricta.

# Otras clases de complejidad relevantes III

## Clase EXPTIME

Problemas resolubles en tiempo exponencial:

$$\text{EXPTIME} = \bigcup_{k \in \mathbb{N}} \text{DTIME}(2^{n^k}).$$

Por el teorema de jerarquía de tiempo,  $P \subsetneq \text{EXPTIME}$  (inclusión estricta demostrada).

## Relaciones conocidas

$$P \subseteq NP \subseteq \text{coNP} \subseteq \text{PSPACE} \subseteq \text{EXPTIME}.$$

No se conoce ninguna inclusión estricta entre  $P$ ,  $NP$  y  $\text{coNP}$ , aunque se conjetura que todas son estrictas.



# Problemas indecidibles: el halting problem I

## Enunciado formal

El problema de la parada (HALT) se define como:

$$\text{HALT} = \{ \langle P, x \rangle \mid P \text{ es un programa que se detiene sobre la entrada } x \}.$$

## Teorema de indecidibilidad (Turing, 1936)

HALT es indecidible: no existe ningún algoritmo que determine correctamente para todo par  $\langle P, x \rangle$  si  $P$  se detiene sobre  $x$ .

# Problemas indecidibles: el halting problem II

## Demostración esquemática por diagonalización

Supongamos que existe un programa  $H$  que decide HALT. Construimos el programa  $D$ :

```
1: function $D(P)$
2: if $H(P, P)$ devuelve "se detiene" then
3: Entra en bucle infinito
4: else
5: Se detiene
6: end if
7: end function
```

¿Se detiene  $D(D)$ ? Si se detiene, entonces  $H(D, D)$  devuelve "se detiene" y  $D(D)$  entra en bucle. Si no se detiene,  $H(D, D)$  devuelve "no se detiene" y  $D(D)$  se detiene. Contradicción.

## Jerarquía de problemas computacionales

- ▶ **Tratables teóricamente:**  $P$  (tiempo polinomial).
- ▶ **Intratables teóricamente pero decidibles:**  $NP$ -completos,  $EXPTIME$  (tiempo exponencial o peor).
- ▶ **Indecidibles:**  $HALT$  y problemas equivalentes (no existe algoritmo general).

# Implicaciones teóricas y prácticas II

## Consecuencias prácticas

- ▶ Los problemas  $NP$ -completos requieren enfoques alternativos: algoritmos aproximados, heurísticas, metaheurísticas (algoritmos genéticos, recocido simulado), o restricción a casos especiales tratables.
- ▶ La indecidibilidad impone límites fundamentales al análisis automático de programas (verificación, detección de errores, optimización).
- ▶ La criptografía moderna se basa en la conjetura  $P \neq NP$ : la seguridad de esquemas como RSA depende de que ciertos problemas (factorización) sean intratables en la práctica.

# Implicaciones teóricas y prácticas III

## Reflexión final

El análisis de complejidad computacional proporciona un marco riguroso para comprender los límites inherentes de la computación. Distingue entre problemas que pueden resolverse eficientemente, aquellos que requieren recursos prohibitivos pero son en principio resolubles, y aquellos que son fundamentalmente irresolubles por medios algorítmicos. Esta comprensión es esencial para el diseño de sistemas computacionales realistas y para establecer expectativas razonables sobre lo que la tecnología puede lograr.

# Ejercicios propuestos

# Ejercicios básicos de notación asintótica I

## Ejercicio 1: Determinar complejidad asintótica

Para cada función, determine su clase asintótica dominante ( $\Theta(\cdot)$ ):

1.  $T(n) = 2n + 10 \log n + 25$
2.  $T(n) = n^3 + 100n^2 + 50$
3.  $T(n) = T(n-1)^2$  (analice el crecimiento)
4.  $T(n) = T(n-1) + T(n-2)$  (relación de Fibonacci)
5.  $T(n) = 2^n + n^3$

# Ejercicios básicos de notación asintótica II

## Ejercicio 2: Ordenar por crecimiento asintótico

Ordene cada conjunto de funciones de menor a mayor tasa de crecimiento:

1.  $\{\log n, n, n^2, 2^n, n!\}$
2.  $\{\sqrt{n}, n \log n, n^3, 3^n\}$
3.  $\{\log \log n, \log n, \sqrt{n}, n\}$
4.  $\{n^2, n^2 \log n, n^3, 2^n\}$
5.  $\{\log n, n^{1/3}, \sqrt{n}, n, n \log n\}$



# Ejercicios básicos de notación asintótica III

## Ejercicio 3: Clasificación asintótica precisa

Para cada función, determine la notación asintótica más ajustada:

1.  $T(n) = 5n^2 + 3n$
2.  $T(n) = n \log n + 10$
3.  $T(n) = 50n + 1000$
4.  $T(n) = 2^n + n^4$
5.  $T(n) = \log n + \sqrt{n}$

# Ejercicios avanzados de notación asintótica I

## Ejercicio 4: Ordenamiento complejo

Ordene de menor a mayor crecimiento asintótico:

$\{\log^* n, \log \log n, \log n, (\log n)^2, \sqrt{n}, n^{\log \log n}, (\log n)^{\log n}, n^{\sqrt{\log n}}, 2^{\sqrt{n}}, 2^n, n!\}$

donde  $\log^* n$  es el logaritmo iterado (número de veces que debe aplicarse  $\log$  hasta obtener  $\leq 1$ ).

# Ejercicios avanzados de notación asintótica II

## Ejercicio 5: Demostraciones formales

Demuestre rigurosamente mediante definiciones formales:

1.  $n^2 + 3n \log n = O(n^2)$
2.  $2^n \neq O(n^k)$  para cualquier constante  $k > 0$
3.  $\log(n!) = \Theta(n \log n)$
4.  $n^{\log n} = o(2^n)$
5.  $\sum_{i=1}^n i^k = \Theta(n^{k+1})$  para  $k > -1$

# Ejercicios de recurrencias lineales

# Ecuaciones homogéneas lineales (básicas) I

## Instrucciones

Resuelva cada recurrencia determinando:

- ▶ Polinomio característico y sus raíces.
- ▶ Forma general de la solución.
- ▶ Constantes mediante condiciones iniciales genéricas  $T(0) = c_0$ ,  $T(1) = c_1$ , etc.

1.  $T(n) - 3T(n-1) + 2T(n-2) = 0$

2.  $T(n) - 5T(n-1) + 6T(n-2) = 0$

3.  $T(n) - 4T(n-1) + 4T(n-2) = 0$

4.  $T(n) - 6T(n-1) + 9T(n-2) = 0$

5.  $T(n) - 2T(n-1) - 3T(n-2) = 0$

6.  $T(n) + T(n-1) - 6T(n-2) = 0$

## Ecuaciones homogéneas lineales (básicas) II

- 7.  $T(n) - 7T(n-1) + 12T(n-2) = 0$
- 8.  $T(n) - 8T(n-1) + 16T(n-2) = 0$
- 9.  $T(n) - T(n-1) - 2T(n-2) = 0$
- 10.  $T(n) + 2T(n-1) - 8T(n-2) = 0$
- 11.  $T(n) - 9T(n-1) + 20T(n-2) = 0$
- 12.  $T(n) - 10T(n-1) + 25T(n-2) = 0$
- 13.  $T(n) - 3T(n-1) - 4T(n-2) + 12T(n-3) = 0$
- 14.  $T(n) - 6T(n-1) + 11T(n-2) - 6T(n-3) = 0$
- 15.  $T(n) - 4T(n-1) + T(n-2) + 6T(n-3) = 0$

# Ecuaciones homogéneas lineales (avanzadas) I

## Ejercicios con raíces complejas y multiplicidades altas

1.  $T(n) + 4T(n-2) = 0, \quad T(0) = 1, T(1) = 0$
2.  $T(n) - 2T(n-1) + 2T(n-2) = 0, \quad T(0) = 1, T(1) = 2$
3.  $T(n) - 3T(n-1) + 3T(n-2) - T(n-3) = 0, \quad T(0) = 1, T(1) = 2, T(2) = 3$
4.  $T(n) - 4T(n-2) = 0, \quad T(0) = 1, T(1) = 2$
5.  $T(n) - 5T(n-1) + 8T(n-2) - 4T(n-3) = 0, \quad T(0) = 1, T(1) = 3, T(2) = 8$

# Ecuaciones homogéneas lineales (avanzadas) II

## Sugerencia para raíces complejas

Si  $r = \alpha \pm \beta i = \rho e^{\pm i\theta}$  con  $\rho = \sqrt{\alpha^2 + \beta^2}$  y  $\theta = \arctan(\beta/\alpha)$ , entonces la contribución real es:

$$\rho^n (A \cos(n\theta) + B \sin(n\theta)) .$$



# Ecuaciones no homogéneas lineales (básicas) I

## Instrucciones

Resuelva mediante método constructivo (homogénea + particular) o polinomio característico extendido. Verifique resonancia cuando el término forzante coincide con una raíz del polinomio homogéneo.

1.  $T(n) - 3T(n-1) + 2T(n-2) = 2^n$
2.  $T(n) - 5T(n-1) + 6T(n-2) = n \cdot 3^n$
3.  $T(n) - 4T(n-1) + 4T(n-2) = 5^n$
4.  $T(n) - 6T(n-1) + 9T(n-2) = n^2 \cdot 3^n$
5.  $T(n) - 2T(n-1) - 3T(n-2) = (-1)^n + 2^n$
6.  $T(n) + T(n-1) - 6T(n-2) = n \cdot 2^n + 3^n$
7.  $T(n) - 7T(n-1) + 12T(n-2) = 4^n + n$
8.  $T(n) - 8T(n-1) + 16T(n-2) = n^2 \cdot 4^n$

## Ecuaciones no homogéneas lineales (básicas) II

- 9.  $T(n) - T(n-1) - 2T(n-2) = 3^n + n \cdot (-1)^n$
- 10.  $T(n) + 2T(n-1) - 8T(n-2) = 2^n + n^2 \cdot 3^n$
- 11.  $T(n) - 9T(n-1) + 20T(n-2) = n \cdot 5^n + 2^n \cdot n^3$
- 12.  $T(n) - 10T(n-1) + 25T(n-2) = n^2 \cdot 5^n + 7^n$
- 13.  $T(n) - 3T(n-1) - 4T(n-2) + 12T(n-3) = 2^n + n \cdot 3^n$
- 14.  $T(n) - 6T(n-1) + 11T(n-2) - 6T(n-3) = n^2 \cdot 2^n + 4^n$
- 15.  $T(n) - 4T(n-1) + T(n-2) + 6T(n-3) = (-2)^n + n^3$

# Ecuaciones no homogéneas lineales (avanzadas) I

## Ejercicios con resonancia múltiple y combinaciones complejas

1.  $T(n) - 2T(n-1) + T(n-2) = n^2, \quad T(0) = 0, T(1) = 0$
2.  $T(n) - 4T(n-1) + 4T(n-2) = n \cdot 2^n, \quad T(0) = 1, T(1) = 3$
3.  $T(n) - 3T(n-1) + 2T(n-2) = (-1)^n + 2^n, \quad T(0) = 1, T(1) = 0$
4.  $T(n) - T(n-1) = n^3 \cdot 2^n, \quad T(0) = 1$
5.  $T(n) - 6T(n-1) + 9T(n-2) = n^2 \cdot 3^n + 2^n, \quad T(0) = 0, T(1) = 1$

## Ejercicio 6: Sistema lineal acoplado

Resuelva el sistema:

$$\begin{cases} A(n) = 2A(n-1) + B(n-1), \\ B(n) = A(n-1) + 2B(n-1), \end{cases} \quad A(0) = 1, B(0) = 0.$$

# Recurrencias no estándar I

## Ejercicio 1: Coeficientes variables

Resuelva:

$$T(n) = n \cdot T(n - 1) + n!, \quad T(0) = 1.$$

# Recurrencias no estándar II

## Ejercicio 2: Divide-y-vencerás no estándar

Resuelva:

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n, \quad T(1) = 1.$$

Demuestre que  $T(n) = \Theta(n \log n)$ .

# Recurrencias no estándar III

## Ejercicio 3: Recurrencia logística discreta (no lineal)

Analice el comportamiento asintótico de:

$$x_{n+1} = rx_n(1 - x_n), \quad x_0 = 0.5,$$

para  $r \in (0, 1) \cup (1, 3)$ .

# Aplicación del teorema maestro

# Teorema maestro: Caso 1 (dominancia de las hojas) I

## Instrucciones

Para  $T(n) = aT(n/b) + f(n)$  con  $d = \log_b a$ :

- ▶ Verifique  $\exists \varepsilon > 0$  tal que  $f(n) = O(n^{d-\varepsilon})$ .
- ▶ Concluya  $T(n) = \Theta(n^d)$ .

1.  $T(n) = 2T(n/2) + n/\log n$
2.  $T(n) = 2T(n/2) + n^{0.9}$
3.  $T(n) = 3T(n/3) + \log n$
4.  $T(n) = 4T(n/2) + n$
5.  $T(n) = 8T(n/2) + n^2$
6.  $T(n) = 9T(n/3) + n \log n$
7.  $T(n) = 3T(n/4) + \log^5 n$



## Teorema maestro: Caso 2 generalizado

$(f(n) = \Theta(n^d \log^k n))$  !

### Clasificación según el valor de $k$

Si  $f(n) = \Theta(n^d \log^k n)$  con  $d = \log_b a$ , entonces:

$$T(n) = \begin{cases} \Theta(n^d \log^{k+1} n) & \text{si } k > -1, \\ \Theta(n^d \log \log n) & \text{si } k = -1, \\ \Theta(n^d) & \text{si } k < -1. \end{cases}$$

## Teorema maestro: Caso 2 generalizado

$$(f(n) = \Theta(n^d \log^k n)) \parallel$$

### Ejercicios con $k > -1$

1.  $T(n) = 2T(n/2) + n$
2.  $T(n) = 2T(n/2) + n \log n$
3.  $T(n) = 2T(n/2) + n \log^3 n$
4.  $T(n) = 4T(n/2) + n^2 \log n$
5.  $T(n) = 3T(n/3) + n \log^2 n$

## Teorema maestro: Caso 2 generalizado

$(f(n) = \Theta(n^d \log^k n))$  III

### Ejercicios con $k = -1$

6.  $T(n) = 2T(n/2) + \frac{n}{\log n}$

7.  $T(n) = 3T(n/3) + \frac{n}{\log n}$

## Teorema maestro: Caso 2 generalizado

$(f(n) = \Theta(n^d \log^k n))$  IV

### Ejercicios con $k < -1$

8.  $T(n) = 2T(n/2) + \frac{n}{\log^2 n}$

9.  $T(n) = 2T(n/2) + \frac{n}{\log^3 n}$

### Ejercicio frontera

$$T(n) = 3T(n/3) + \frac{n}{\log \log n}$$

# Teorema maestro: Caso 3 (dominancia de la raíz) I

## Instrucciones

Para  $T(n) = aT(n/b) + f(n)$  con  $d = \log_b a$ :

- ▶ Verifique  $\exists \varepsilon > 0$  tal que  $f(n) = \Omega(n^{d+\varepsilon})$ .
- ▶ Verifique condición de regularidad:  $\exists c < 1, n_0$  tal que  $af(n/b) \leq cf(n)$  para  $n \geq n_0$ .
- ▶ Concluya  $T(n) = \Theta(f(n))$ .

1.  $T(n) = 2T(n/2) + n^{1.1}$
2.  $T(n) = 3T(n/3) + n^{1.2}$
3.  $T(n) = 4T(n/2) + n^{5/2}$
4.  $T(n) = 9T(n/3) + n^{2.1}$
5.  $T(n) = 5T(n/5) + n^{1.01}$

## Teorema maestro: Caso 3 (dominancia de la raíz) II

6.  $T(n) = 2T(n/4) + \sqrt{n} \log n$

7.  $T(n) = 3T(n/9) + n^{0.6}$

# Teorema maestro: ejercicios avanzados I

## Cambio de variable para argumentos no lineales

Para  $T(n) = aT(n^{1/b}) + f(n)$ , defina  $m = \log_2 n$  y  $S(m) = T(2^m)$ . Entonces:

$$S(m) = aS(m/b) + f(2^m),$$

a la cual se aplica el teorema maestro estándar.

1.  $T(n) = 2T(\sqrt{n}) + \log n$
2.  $T(n) = T(\sqrt{n}) + 1$
3.  $T(n) = 3T(\sqrt{n}) + n^{1/4}$
4.  $T(n) = 2T(\sqrt[4]{n}) + \log^2 n$
5.  $T(n) = 5T(\sqrt{n}) + \sqrt{n}$

# Teorema maestro: ejercicios avanzados II

## Recurrencias fuera del alcance del teorema maestro estándar

6.  $T(n) = T(n/2) + T(n/4) + T(n/8) + n$

7.  $T(n) = 2T(n/2) + \frac{n}{\log^* n}$

8.  $T(n) = 3T(n/2) + n \log n$

9.  $T(n) = T(n/2) + T(n/3) + n$



# Problemas de aplicación práctica I

## Ejercicio 1: Análisis de Quicksort con pivote mediano

El algoritmo Quicksort con pivote mediano satisface en promedio:

$$T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + cn, \quad T(0) = T(1) = \Theta(1).$$

Demuestre que  $T(n) = \Theta(n \log n)$ .

# Problemas de aplicación práctica II

## Ejercicio 2: Algoritmo de Strassen optimizado

Una variante del algoritmo de Strassen satisface:

$$T(n) = 6T(n/2) + \Theta(n^2).$$

1. Determine la complejidad.
2. Compare: estándar  $\Theta(n^3)$ , Strassen original  $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$ , esta variante.

# Problemas de aplicación práctica III

## Ejercicio 3: Búsqueda en BST desequilibrado

Peor caso (árbol degenerado):

$$T(n) = T(n-1) + \Theta(1), \quad T(1) = \Theta(1) \Rightarrow T(n) = \Theta(n).$$

Caso promedio en BST aleatorio:

$$T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + \Theta(1).$$

Demuestre por inducción que  $T(n) = \Theta(\log n)$ .

# Teorema maestro: Caso 3 (dominancia de la raíz) I

## Instrucciones

Para  $T(n) = aT(n/b) + f(n)$  con  $d = \log_b a$ :

- ▶ Verifique  $\exists \varepsilon > 0$  tal que  $f(n) = \Omega(n^{d+\varepsilon})$ .
- ▶ Verifique condición de regularidad:  $\exists c < 1, n_0$  tal que  $af(n/b) \leq cf(n)$  para  $n \geq n_0$ .
- ▶ Concluya  $T(n) = \Theta(f(n))$ .

1.  $T(n) = 2T(n/2) + n^{1.1}$
2.  $T(n) = 3T(n/3) + n^{1.2}$
3.  $T(n) = 4T(n/2) + n^{5/2}$
4.  $T(n) = 9T(n/3) + n^{2.1}$
5.  $T(n) = 5T(n/5) + n^{1.01}$

## Teorema maestro: Caso 3 (dominancia de la raíz) II

6.  $T(n) = 2T(n/4) + \sqrt{n} \log n$

7.  $T(n) = 3T(n/9) + n^{0.6}$

# Teorema maestro: ejercicios avanzados I

## Cambio de variable para argumentos no lineales

Para  $T(n) = aT(n^{1/b}) + f(n)$ , defina  $m = \log_2 n$  y  $S(m) = T(2^m)$ . Entonces:

$$S(m) = aS(m/b) + f(2^m),$$

a la cual se aplica el teorema maestro estándar.

1.  $T(n) = 2T(\sqrt{n}) + \log n$
2.  $T(n) = T(\sqrt{n}) + 1$
3.  $T(n) = 3T(\sqrt{n}) + n^{1/4}$
4.  $T(n) = 2T(\sqrt[4]{n}) + \log^2 n$
5.  $T(n) = 5T(\sqrt{n}) + \sqrt{n}$

# Teorema maestro: ejercicios avanzados II

## Recurrencias fuera del alcance del teorema maestro estándar

6.  $T(n) = T(n/2) + T(n/4) + T(n/8) + n$

7.  $T(n) = 2T(n/2) + \frac{n}{\log^* n}$

8.  $T(n) = 3T(n/2) + n \log n$

9.  $T(n) = T(n/2) + T(n/3) + n$

# Problemas de aplicación práctica I

## Ejercicio 1: Análisis de Quicksort con pivote mediano

El algoritmo Quicksort con pivote mediano satisface en promedio:

$$T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + cn, \quad T(0) = T(1) = \Theta(1).$$

Demuestre que  $T(n) = \Theta(n \log n)$ .



# Problemas de aplicación práctica II

## Ejercicio 2: Algoritmo de Strassen optimizado

Una variante del algoritmo de Strassen satisface:

$$T(n) = 6T(n/2) + \Theta(n^2).$$

1. Determine la complejidad.
2. Compare: estándar  $\Theta(n^3)$ , Strassen original  $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$ , esta variante.

# Problemas de aplicación práctica III

## Ejercicio 3: Búsqueda en BST desequilibrado

Peor caso (árbol degenerado):

$$T(n) = T(n-1) + \Theta(1), \quad T(1) = \Theta(1) \Rightarrow T(n) = \Theta(n).$$

Caso promedio en BST aleatorio:

$$T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + \Theta(1).$$

Demuestre por inducción que  $T(n) = \Theta(\log n)$ .

# Bibliografía recomendada

# Bibliografía recomendada I

## Textos fundamentales

- ▶ Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3<sup>a</sup> ed.). MIT Press.
- ▶ Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Addison-Wesley.
- ▶ Sipser, M. (2013). *Introduction to the Theory of Computation* (3<sup>a</sup> ed.). Cengage Learning.
- ▶ Dasgupta, S., Papadimitriou, C. H., & Vazirani, U. V. (2008). *Algorithms*. McGraw-Hill.
- ▶ Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3<sup>a</sup> ed.). Addison-Wesley.

# Bibliografía recomendada II

## Recursos complementarios y software

### ► Visualización algorítmica:

Visualgo: <https://visualgo.net>

Algorithm Visualizer: <https://algorithm-visualizer.org>

### ► Entornos computacionales libres:

Python (SciPy/NumPy): <https://www.python.org>

SageMath: <https://www.sagemath.org>

GNU Octave: <https://www.gnu.org/software/octave>

### ► Referencias teóricas en línea:

Complexity Zoo: <https://complexityzoo.net>

CLRS Solutions: <https://walkccc.me/CLRS>

# Bibliografía recomendada III

## Recomendación pedagógica

Para el estudio autónomo se sugiere:

1. Leer los capítulos correspondientes de Cormen *et al.* como referencia principal.
2. Complementar con ejemplos prácticos en Python/SageMath para consolidar conceptos.
3. Utilizar Visualgo para desarrollar intuición sobre el comportamiento de algoritmos clásicos.
4. Resolver sistemáticamente los ejercicios propuestos, comenzando por los casos básicos antes de abordar problemas avanzados.



universidad  
de león

<https://www.unileon.es/>